

Dashboard and Racing Telemetry

Senior Design Project Final Report

Design Team 08

Cole Barach

Jacob Koshel

Matthew Sullivan

Ethan Zifzal

Anindya Bijoy Das

05/03/2026

Table of Contents

1. Problem Statement	5
1.1 Need (EZ)	6
1.2 Objective (MS)	6
1.3 Background (EZ, MS, CB)	6
1.3.1 General Background	7
1.3.2 Existing Patents	8
1.3.3 Existing Products	9
1.4 Marketing Requirements (MS, CB)	12
2 Engineering Analysis	13
2.1 Design Decisions	13
2.1.1 CPU Selection (CB)	13
2.1.2 Hardware Architecture (EZ):	14
2.1.3 Software Architecture (CB):	18
2.2 Design Calculations (EZ):	18
3. Engineering Requirements Specification (CB, EZ, JK, MS)	19
4. Engineering Standards Specification	20
4.1 Hardware Standards (EZ):	21
4.2 Software Standards (CB):	21
5. Accepted Technical Design	22
5.1 Hardware Design (CB, EZ, JK, MS):	23
5.2 Software Design (CB):	34
5.2.1 The Graphical User Interface	39
5.2.2 The Data Logging Application	44
5.2.3 The Init System	47
5.3 Lessons Learned (CB, EZ)	48
5.4 Financial Budget (EZ)	49
6. Mechanical Sketch (EZ, CB):	50
7. Team Information	54
8. Project Schedules (CB, EZ, JK, MS)	55
9. Conclusions and Recommendations (CB)	65
10. References	66
11. Appendices	68
Appendix A	69

Appendix B	69
------------------	----

List of Figures

Figure 1	11
Figure 2	12
Figure 3	16
Figure 4	16
Figure 5	17
Figure 6	24
Figure 7	25
Figure 8	25
Figure 9	27
Figure 10.....	29
Figure 11	30
Figure 12	31
Figure 13	32
Figure 14	33
Figure 15	33
Figure 16	34
Figure 17	35
Figure 18	36
Figure 19	42
Figure 20	43
Figure 21	44
Figure 22	45
Figure 23	47
Figure 24	49
Figure 25	52
Figure 26	53
Figure 27	53
Figure 28	54
Figure 29	54
Figure 30	55
Figure 31	56
Figure 32	57
Figure 33	57
Figure 34	58

List of Tables

Table 1	15
Table 2	17
Table 3	19
Table 4	19
Table 5	19
Table 6	21
Table 7	24
Table 8	26
Table 9	28
Table 10	29
Table 11	29
Table 12	29
Table 13	30
Table 14	31
Table 15	32
Table 16	33
Table 17	33
Table 18	37
Table 19	38
Table 20	39
Table 21	39
Table 22	40
Table 23	41
Table 24	46
Table 25	48
Table 26	51
Table 27	59

Abstract

Zips Racing is a student design team that develops electric formula-style race cars at the University of Akron. The goal of this design project is to design and manufacture a single device that acts as both a dashboard and data logger for Zips Racing's 2026 vehicle. As a dashboard, the device must be able to display critical data to the driver in real-time. As a data logger, the device must record all data available to it for future analysis. This device will incorporate both custom hardware and software, specifically an LCD display for the dashboard, two CAN 2.0B interfaces for accessing data from the vehicle's other electrical subsystems, internal storage for logging data to, and a central processor for rendering the device's GUI and performing data logging. Mechanically this device must be small enough fit on the front face of the vehicle's cockpit. Electrically, the device must directly interface with the vehicle's low voltage systems. Further, the device's software must be flexible enough for the device to be used on newer vehicles developed by Zips Racing.

1. Problem Statement

1.1 Need (EZ)

The dashboard, datalogging, and data analysis software currently used by Zips Racing rely on costly proprietary hardware and software developed by MoTeC. According to the team captain, the current vehicle's datalogger and dashboard have been passed through the previous 10 vehicles due to the high-cost relative to the team's operational budget. This is a problem, as it means no two vehicles can be operational at the same time, limiting the team's ability to directly compare their performance. As this is a major step in the design process, there is significant value in solving this problem. Moreover, the data analysis tools provided by MoTeC are behind additional paywalls, requiring licensing to obtain the vehicle data in a form compatible with other, more robust, software. A low-cost system capable of replacing the functionality of the MoTeC systems is needed to address these issues.

1.2 Objective (MS)

The objective of this project is to design and manufacture a single device that acts as both a vehicle dashboard and data logger. As a dashboard, the device must be able to display critical data to the driver in real-time. As a data logger, the device must record all data available to it for future analysis. To ensure it does not become deprecated by future changes, the system should be as modular as possible. This modularity extends to the device's software configuration, which will likely require the most changes vehicle-to-vehicle. A potential extension to the device's scope would be the ability to replace the MoTeC systems of previous vehicles with instances of the device to allow them to be tested again.

1.3 Background (EZ, MS, CB)

1.3.1 General Background

The Zips Racing student design team is seeking to move from a pair of proprietary dashboard and data logging devices to an open-source and cost-effective custom design. To gauge the complexity of such a task, existing patents and research papers on both custom and commercial dashboards were analyzed. Additionally, a previously used proprietary data logger was disassembled and reverse engineered to gain an understanding of its architecture.

To make the system affordable, the device's central controller will likely be implemented using an embedded system or a system-on-chip, depending on both packaging constraints and software complexity. The use of embedded systems in automotive applications has gained traction due to their affordability and adaptability. Beneditti et al. (2020)¹ present an example of such an application in their development of a custom dashboard. Their system utilizes a Raspberry Pi 3B, a mass produced and affordable single board computer, well capable enough to drive the display. A custom graphical user interface displays relevant vehicle information to the driver by parsing information from the vehicle's CAN bus.

In the world of racing, data analysis is incredibly important. From making design decisions, giving drivers feedback, and improving vehicle performance, data is a necessity. There is a vast amount of information available in an electric vehicle, from the battery management system's per-cell sensors to the damping position of the suspension. All these signals can't be routed with unique hardware due to the scale of the systems, rather a communication bus is shared by the vehicle's systems. Zips Racing primarily uses the CAN 2.0B standard for inter-system communication. In a system where milliseconds can make the difference between winning and losing, real time data streaming is an important factor. Rohit and Jagrut² (Nimmala & Nimmala, 2024) comment on this importance, discussing the use of higher-level software for automating data analysis. Software like Alteryx uses machine learning to identify correlations between complex data sets. For the case of a crucial race, on the fly results are important which leads to the need to receive, display, and finally store data for future analysis.

As nearly all information within the vehicle is communicated using CAN bus, a data logger must support this protocol. Györök and Beszédes³ (2025) discuss the process of writing

CAN messages to digital storage, such as a USB flash Drive or microSD card. Of course, each device has its own interfaces of communication like Single Level Cell, Multi Level Cell, etc. In a few steps the process to initiate stored data gets complex between CAN language and storage language; nevertheless, CAN will be stored as a raw format which is not readily readable. The file will be translated into a readable format with a CAN DBC file. Brewu⁴ (2022) outlines a custom data logger built upon both an STM32 and ESP32 microcontroller. The data logger portion of the dash will be focused on a similar system. Our system will contain a central processor which reads CAN bus data and stores said data to non-volatile storage.

1.3.2 Existing Patents

There are several existing patent systems that are relevant to our design project. The first patent is a vehicle dashboard primarily aimed at motorsport applications. This design, created by Drew and Herron⁵ (2010) details a data acquisition and display system that connects to a vehicle's on-board computer(s). The goal of this system is what Zips Racing is aiming to achieve with their design. There are, however, some distinct differences between this patent and the system Zips Racing needs. First, Drew and Herron's⁵ (2010) patent are designed for commercial vehicles, as they mention the dashboard will be mounted on the interior of a vehicle. Zips Racing designs an open cockpit vehicle, which requires stricter mechanical criteria, such as ingress protection for driving in the rain and during the rain test that the vehicles are put through. Their patent also briefly mentions data storage and recollection; however, it simply discusses the means of storing data, not analysis of said data. The patent describes storing data points along with a timestamp, which allows for a degree of analysis, but does not touch on resampling or adjusting the time-base, which would be required to compare multiple different signals. Further, the patent does not address the computer software that would be responsible for viewing the logged data. For Zips Racing, this software is just as important as the data logger itself.

Another patent relevant to our design is Damiani, Deregibus, and Dosio's⁶ (2003) patented design. They describe a device featuring a pair of displays which can be navigated using a trackball. Each display can communicate relevant vehicle information. This patent is mainly aimed towards the manipulation of the graphical user interface, such as changing information size, color, and style. This patent relies on older technologies, such as the inclusion of a trackball

as a user interface as opposed to Drew and Herron's⁵ (2010) patent stating that the dashboard display will feature a touch screen.

1.3.3 Existing Products

One product relevant to our design project is the MoTeC Advanced Central Logger (ACL), a commercially available CAN bus data logger developed for use in vehicles. While it has since been obsoleted by newer MoTeC products, the general system functionality and software tools have remained consistent. Zips Racing has used the ACL as their data logger for roughly 11 years, due to its flexibility and reliability. For this very same reason, NASA had selected the ACL for use in the Cockpit Display System of the X-57 Maxwell aircraft. Curry et al.⁷ (2020) claim the key reason MoTeC products were selected was their rugged design. They tested the ACL and its complementary products over a wide temperature range, intense vibration, and at an extreme equivalent altitude. While an aircraft is a much more extreme environment than an electric vehicle, the conditions the device is subjected to are very similar, the most notable being EMI, vibration, and humidity.

The ACL can be configured with a set of CAN signals to be logged. From said CAN signals, extrapolated signals and starting / stopping conditions can be specified. While in the idle state, the device will parse every received CAN message that contains a known signal, checking for the start conditions. Upon the starting conditions being satisfied, every subsequent CAN message will be parsed and logged to the device's flash memory, along with a timestamp. When the stop condition is detected, the device will re-enter the idle state. After the logging session is completed, the data can be pulled from the device via its ethernet interface. A high level diagram of the ACL and its functionality is shown in *Figure 1*.

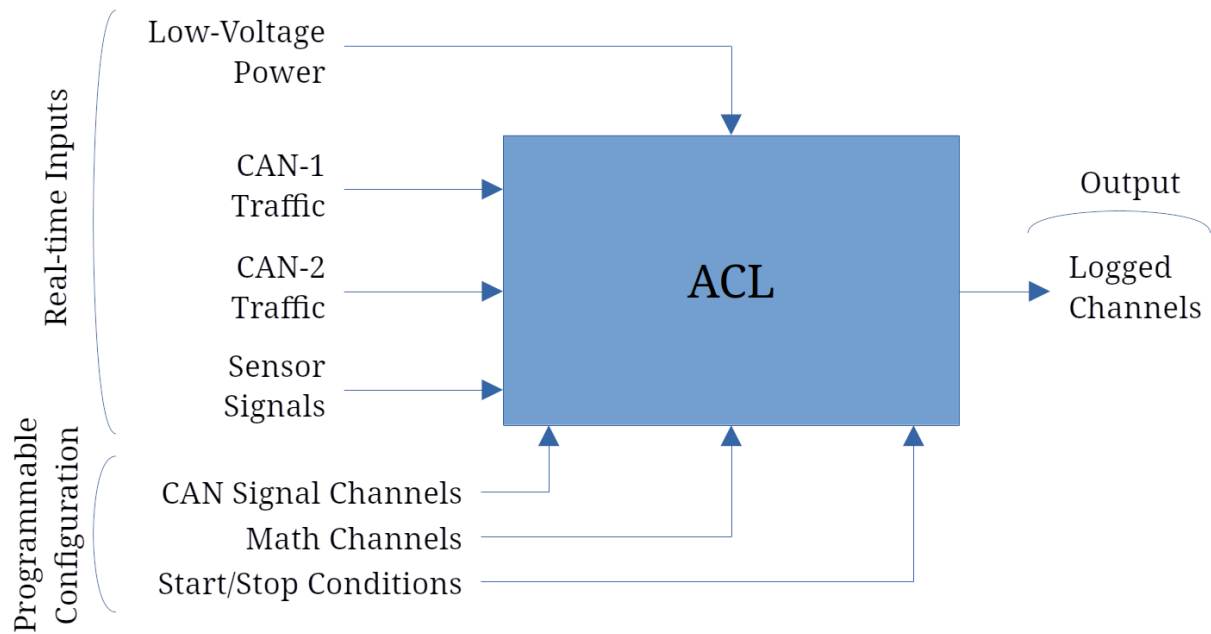


Figure 1: Functional block diagram of the MoTeC ACL.

In order to get an understanding of the how the ACL was implemented, a teardown and reverse-engineering was performed on the ACL owned by Zips Racing. The main purpose of this was to get a high-level understanding of the device's architecture, not to acquire specific details of its operation. From the teardown, it was determined that the main components of the device are the MCP5200 processor, the MT46V16M16 SRAM, the TE28F256J3C non-volatile memory, and the PCI1520 PCI controller. In addition to these components, several notable transceivers for the device's various interfaces were identified. While purely speculative, a general architecture of the system was designed based on the purposes of each component as well as the interfaces that they feature. This design is shown in *Figure 2*.

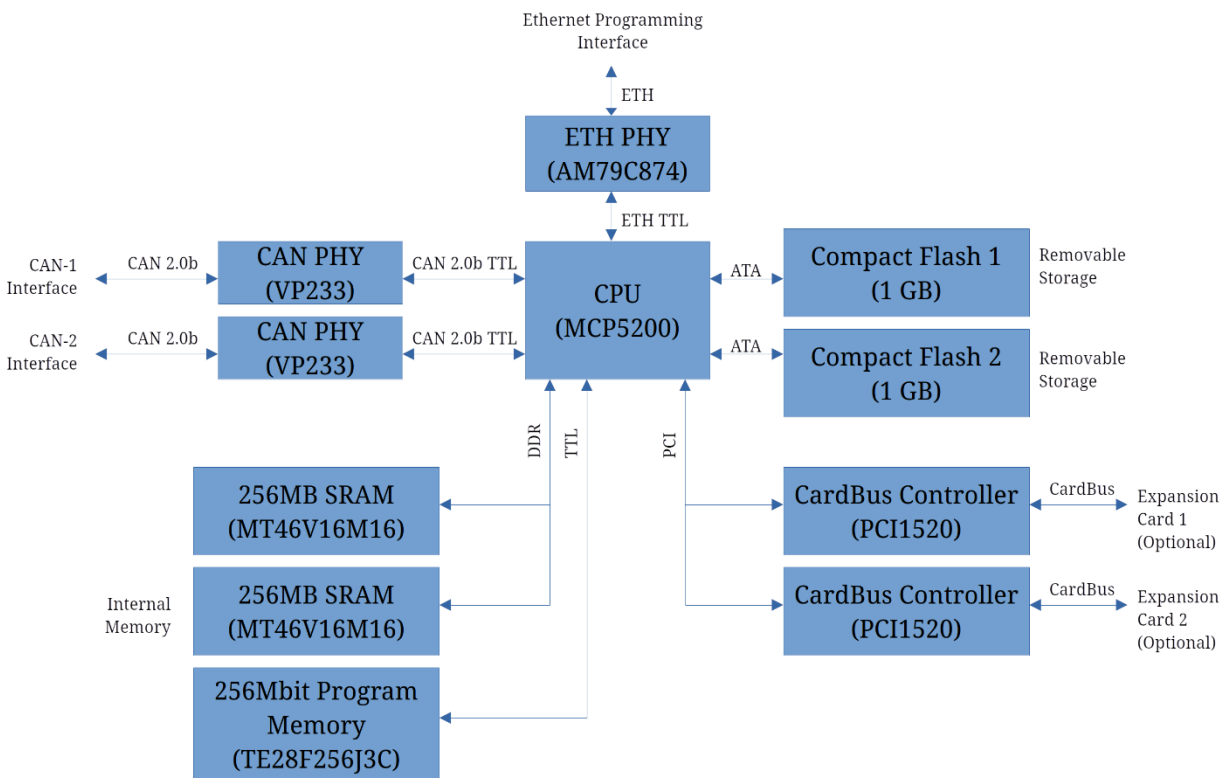


Figure 2: Speculative system architecture of the MoTeC ACL. This diagram includes all notable interfaces and functionality of the device, including the 2 CAN interfaces, the ethernet interface, and the 2 flash storage devices.

Due to its age, the ACL's limitations have started to become apparent. The most notable of which is the device's logging capacity of 1GB, which, due to limited documentation and the product's obsolescence, cannot be upgraded. In a relatively high-load scenario of 2 CAN 2.0A busses, each at 1Mbps and 50% capacity, the total amount of logging time is just under 4 hours. While at a glance this figure seems plenty sufficient, Zips Racing frequently has testing days that run up to 6 or 7 hours. While the problem can be avoided by periodically pulling the data to a laptop, hours of testing data can be lost if somebody forgets to do so.

$$T_{\text{Logging}} = \frac{1 \text{ GB}}{2 * 50\% * 1 \text{ Mb/s} * \frac{8 \text{ B}}{114 \text{ b}^{(8)}}} = 3 \text{ h}, 57 \text{ min}, 30 \text{ s}$$

8. Using 114 bits as the minimum 8-byte frame size, as specified in "Introduction to the Controller Area Network (CAN)" by Steve Corrigan⁸. From page 4, assuming no bit stuffing, a minimum of 114 bits are required for an 11-bit ID, 8-byte CAN frame.

Another issue with the ACL is that of software compatibility. As both the hardware and software of ACL is completely proprietary, the device's functionality is limited by what the software allows the user to do. As a result of this, many superficial problems have arisen and have made working with the device difficult. A notable example is how the ACL's software manages CAN databases. The de facto standard for how to interpret CAN bus traffic is the CAN DBC file format, a human-readable file format developed by Vector. Most modern CAN software uses this format, including all software developed by Zips Racing. Due to its age, the MoTeC ACL Manager software does not support the CAN DBC format, meaning 2 completely independent files detailing the exact same thing must be developed and maintained.

1.4 Marketing Requirements (MS, CB)

1. The device must be significantly cheaper than the previous MoTeC systems.
2. The device must display real-time information to the driver via an intuitive GUI.
3. The device must be compatible with the vehicle's electrical systems.
4. The device's mechanical footprint must be minimized as to not impede the surrounding design of the vehicle.
5. Logged data must be accessible in a standard format for compatibility with other data analysis and simulation software.
6. The design should promote flexibility and maintainability so the device can be reused and repurposed for future vehicles.

2 Engineering Analysis

2.1 Design Decisions

2.1.1 CPU Selection (CB)

One of the biggest decisions that had to be made was what type of CPU the unit should use. The two biggest choices for the CPU were either a System-on-Module, System-on-Chip, or a Microcontroller. For the microcontroller, the graphical processing unit can be either on-board the microcontroller or onboard the display.

For choosing a CPU the decision was broken down into 5 main criteria. The highest ranked criteria were adaptability / maintainability and software simplicity. The driving reason for this decision is the fact that the project is expected to be used for several years, meaning the software must be able to be updated to meet new requirements and match changes to the vehicle's electrical systems. Hardware simplicity and manufacturability were the next two criteria, as being able to modify and adapt the electrical design is important for future proofing the project. Lastly the mechanical footprint was the lowest ranking criterion, as the board shape is largely determined by the size of the unit's screen, not the largest board component.

With these criteria, a decision matrix was made for the 4 options. From this, the system-on-module was determined to be the best option. This matrix is shown in *table 1*.

Table 1: Decision matrix used for determining CPU selection.

	Weight	System-on-Module	System-on-Chip	Microcontroller & Display with Onboard GPU	Microcontroller with Onboard GPU
Adaptability / Maintainability	4	9	4	4	4
Software Simplicity	3	6	5	3	1
Hardware Simplicity	2	3	1	8	8
Manufacturability	2	5	1	8	8
Mechanical Footprint	1	5	4	7	10
Score	12	73	50	64	52

2.1.2 Hardware Architecture (EZ):

Due to restrained mechanical packaging of the dashboard, component selection had to be carefully selected to reduce the vertical height of the PCB to minimize the thickness of the device. To this end, surface mount devices were used where possible to reduce the physical size of components and most non-power passives were selected to the 0603 size to minimize footprint while still being easily solderable. Additionally, steps were taken to consolidate components to reduce their overall size. The CAN controller and transceiver are examples of this as the MCP25625 was selected to implement the CAN controller and CAN transceiver circuits to create each of the CAN Interfaces for the device. This chip combines the functionality of the MCP2515 and MCP2562 CAN controller and receiver onto a singular IC. The schematic of the circuit was drawn in Altium Designer and placed in the CAN interface hierarchical block. The datasheet for the MCP25625 is referenced in *appendix A*.

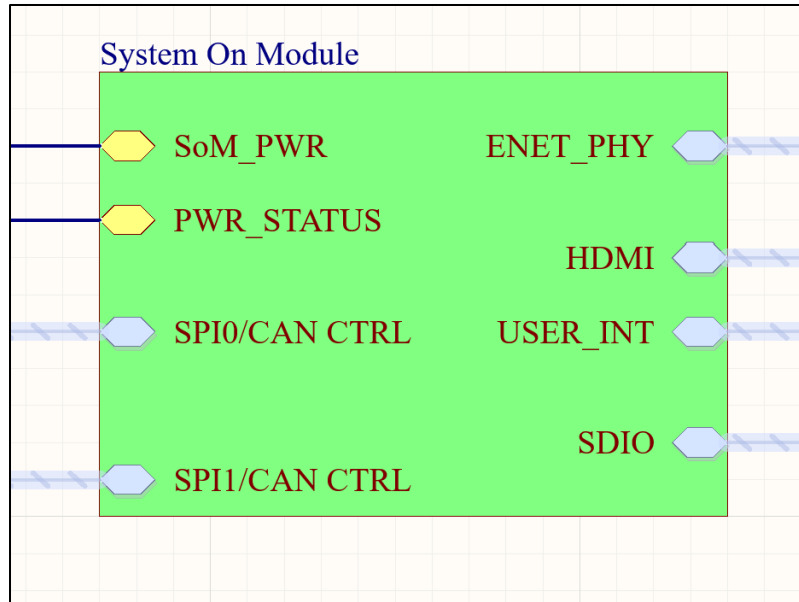


Figure 3: Example of hierarchical design block in Altium Designer

A hierarchical schematic system was implemented in Altium Designer with the aim of improving the organization and readability of the project by breaking the design into its functional components. Furthermore, utilizing sub-sheets and classes within the software, PCB layout can be reused for duplicated circuits in addition to creating barriers for certain signals within the PCB layout. *Figure 3*, provides an example of what a lower-level schematic block looks like within Altium Designer. This system of design also allows for greater collaboration as the schematic sub-sheets can be modified independently so multiple people can work on the project simultaneously.

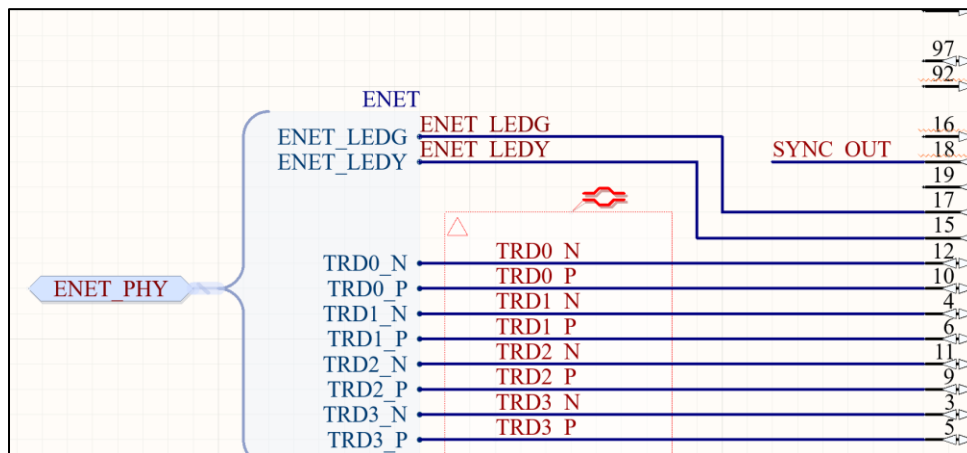


Figure 4: Implementation of differential pair directive in Altium Designer

In the design, differential pair classes were used in Altium for HDMI, USB, and Ethernet interfaces to ensure high-speed signal integrity. Differential signals carry complementary data on two tightly coupled traces, allowing the receiver to detect the voltage difference rather than the absolute voltage, which reduces noise and crosstalk. *Figure 4* provides an illustration of the differential pair classes in Altium designer. To define a pair, the differential signals are appended by “_N” or “_P”, and a differential pair directive is placed on both signals. By defining differential pair classes, we were able to enforce controlled impedance, matched trace lengths, and consistent routing, all of which are critical for maintaining data integrity at high speeds.

The Compute Module datasheet provides the required impedance listed in *Table 2*, length matching in a pair, and length matching information between pairs seen in *Table 3*, which was implemented in the board design. The physical realization of length matching can be seen in *Figure 5* where the “squares” added to each of the nets allows both signals to be of equal length. These are created using the differential pair routing tool which automatically matches pair length to the desired specifications.

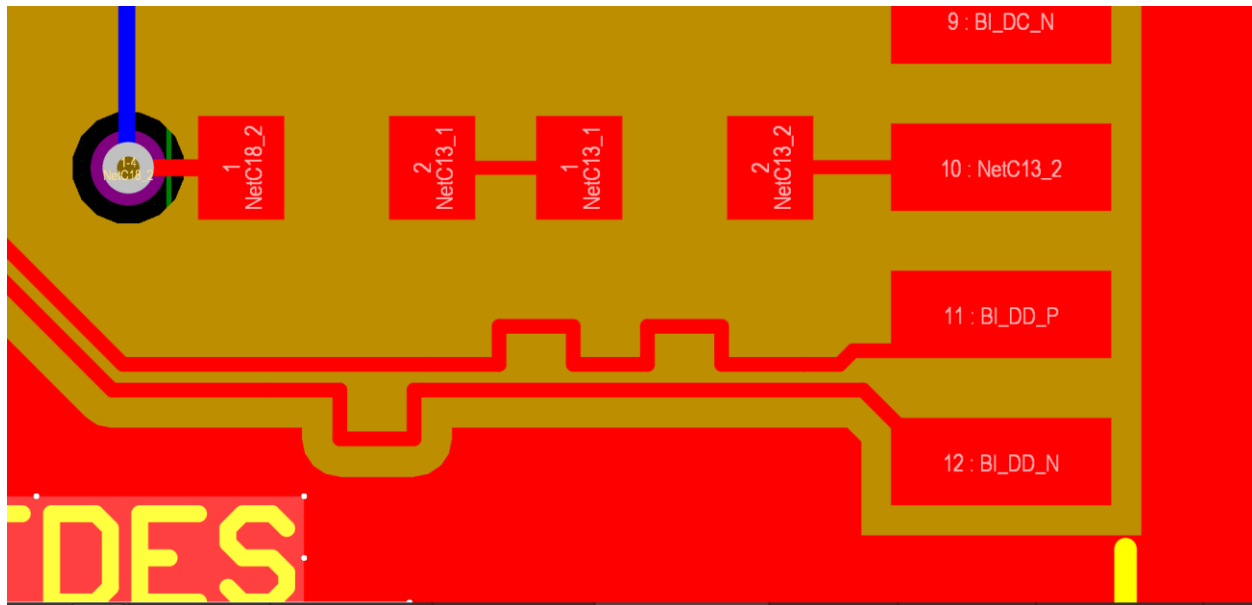


Figure 5: Ethernet pair length matching

Table 2: Differential pair impedances

Signals	Impedance (Ω)
Ethernet	100
HDMI	100
USB	90

There are two ways to configure differential pairs in Altium Designer's PCB editor using either the layer stack manager or the design rules manager. Since JLCPCB was the manufacturer of choice, the design rules manager method was chosen since JLCPCB provides impedance information for their specific impedance-controlled stack-ups. Their stack-up data could simply be placed in a differential pair class within the design rules manager for HDMI, USB, and Ethernet. A four-layer board with an impedance profile where 100Ω traces were larger than 5 mil was selected as additional manufacturing costs are applied to circuit boards with traces smaller than 5 mil. Tables 4 and 5 detail the specifications for the selected stack-up.

Table 3: Impedance length matching requirements

Signal	Length Match	Pair Match
HDMI	<0.15mm	<25mm
Ethernet	<0.15mm	<50mm

Table 4: JLCPCB specified trace widths selected stack-up

Impedance (Ω)	Trace Width (mil)	Trace Spacing (mil)
90	6.6	10
100	5.24	10

Table 5: JLCPCB stack-up specifications

Layers	4
Thickness	1.6mm
Inner Copper Weight	0.5oz
Outer Copper Weight	1oz

2.1.3 Software Architecture (CB):

Due to the complexity of the system's software, quite a bit of forethought had to go into the software architecture. A monolithic architecture was ruled out quite early, as such a design would make unit testing and isolating non-portable code difficult. After some back-and-forth, a system of 3 core applications was landed on: the graphical user interface (GUI), the data logger, and the init-system. With this design, the GUI and data logger are completely independent and portable, meaning development can be done independent of the system's hardware. The init-system is responsible for integrating the other 2 applications, and will thus be the last component developed. The responsibilities and details of these applications will be discussed in the *Engineering Standards Specification* and the *Accepted Technical Design* sections.

2.2 Design Calculations (EZ):

The regulator output current rating was selected by adding the current necessary for both the display and compute module on their respective 5 V power rail. The display's datasheet indicates to design for a max current of 1A at the highest brightness and the compute module draws roughly 1A at a moderate load. Each device receives a separate 5 V rail powered by two

DC/DC converters to reduce the capacitor size required by the system on module during power down. The 5 V rail for the compute module contains a converter powered by a bulk capacitance rail charged to the source voltage of the system. This creates a holdup circuit where a Schottky diode blocks the capacitors from back feeding the source under power down.

$$E = \frac{1}{2}(3844 \mu F)(24 V)^2 = 1.11 J$$

Under use with custom software running, the compute module draws roughly 3.5 W under load and the required 200 ms to safely power down. Assuming an efficiency of 92% from the converter which is a conservative estimate from the datasheet the time provided by the holdup circuit can be approximated to be roughly 291 ms.

$$t = \frac{E * 92\%}{3.5 W} = 0.291 s$$

3. Engineering Requirements Specification (CB, EZ, JK, MS)

Table 6: Engineering Requirement Table.

Req. No.	Marketing Req.	Engineering Requirement	Justification
1	1	The cost to manufacture multiple devices shall not exceed \$1000 per unit.	The team operates on a fixed budget.
2	2	The device must feature a programmable graphical user interface that can display signals from CAN busses in real-time.	The dashboard must provide feedback to the driver in real-time.
3	2	The GUI must include: - Vehicle velocity - Accumulator voltage - Max accumulator temperature - Max inverter temperature	Velocity, HV voltage, and temperature are important parameters to monitor during endurance racing.
4	3	The device must have at least 2 CAN 2.0B interfaces for receiving data from the vehicle's systems.	All information to be logged is transmitted on the vehicle's CAN busses.
5	3	The device must operate normally over the supplied voltage range of 11V to 28V.	The vehicle's GLV systems operate on either 12V or 24V.
6	3	The device must not consume more than 15W of power.	The vehicle has a limited battery capacity.
7	4	The device shall weigh no more than 600g.	Weight is one of the most important parameters to optimize for in FSAE vehicles.
8	4	The device shall be no more than 4.25" tall or 7" wide, nor should it protrude 1.25" further from the mounting plane.	The device's footprint is limited by the surrounding design of the chassis.
9	5	The device must log all received CAN messages to internal storage for subsequent analysis.	The device is intended to be used as a data logger.
10	5	The device shall allow the user to digitally retrieve the logged data.	Retrieving data must be quick and convenient analysis.

4. Engineering Standards Specification

4.1 Hardware Standards (EZ):

The device incorporates multiple hardware communication standards to allow for reliable operation. For high-speed data transfer between the display and the SoM, it supports USB Hi-speed which complies with the USB 2.0 standard and will be used to handle touch screen data. While the SoM supports USB 3.0, it is not necessary to use it as there will be no benefit from the increased complexity. Video output from the SoM will utilize HDMI 1.4 which defines the standard for high-definition video transmission. For network connectivity, the device adheres to the IEEE 802.3 standard, enabling Ethernet LAN communication between the device and a user's computer for uploading data. Vehicle communication is facilitated through the ISO 11898 standard, which specifies the data link and physical layer for Controller Area Network (CAN) bus, allowing the device to read vehicle CAN bus data. Additionally, the SPI protocol is implemented for serial communication between the SoM and the CAN controller. Display brightness is controlled using Pulse Width Modulation (PWM), a widely adopted technique for power regulation which will allow control over the brightness levels of the display.

4.2 Software Standards (CB):

The C programming language, specifically GNU 11, was chosen as the primary programming language of this project. The key reason for this decision was the language's efficiency, which will likely be needed for high throughput data logging. Another benefit to using C is the language's tight integration with the POSIX specification. As embedded Linux is being used for the operating system, the POSIX standards are the best technical reference available. POSIX defines the system calls, user-space libraries, and utility applications of the OS.

For extending the capabilities of the OS, the Linux kernel provides the kernel module interface. The SocketCAN kernel module defines a standardized interface for working with CAN busses on Linux. Based on the Berkley socket API, this module extends the POSIX-defined Socket interface of the Linux kernel with a new protocol family. Similar extensions exist for GPIO and PWM hardware, being the GPIO subsystem and PWM subsystems, respectively.

For the data logging application of the device, the ASAM MDF standard is the most significant. Acting somewhat as a de facto standard for automotive data logging, this standard defines the MDF file format, identified by the .mf4 file extension. The MDF file is a binary

record-based file format, intended for recording either physical or extrapolated measurements that vary with time. The MDF Bus Logging Specification is a corollary standard defining how the MDF format can be used to log raw information from a variety of communication, including CAN bus. This standard is used by a wide variety of automotive data analysis software, for example, MATLAB and ASAMMDF-GUI. By implementing the MDF standard, the data logger allows users to use said 3rd party software, lifting the burden of developing said data analysis software off of this project. While the MDF standard is not publicly available, a list of public resources has been provided in *appendix A*.

For the GUI subsystem of the device, the two most significant standards are those used for the implementation of the front-end and back-end of the application. The back-end is built around the CAN DBC standard. This de facto standard is used for documenting and defining the signals in a CAN bus. Identified by the .dbc file extension, this format defines the contents, origin, and interpretation of every CAN message in a CAN bus. By being able to parse a CAN DBC file programmatically, the GUI can be quickly adapted to fit changes to the vehicle's electrical systems. While an official standard is not publicly available, a list of public resources has been provided in *appendix A*. For the front-end of the GUI, the GTK library was chosen. Similar to the choice of programming language, GTK was chosen for its relatively lightweight nature.

5. Accepted Technical Design

5.1 Hardware Design (CB, EZ, JK, MS):

At a high level, the core responsibilities of the device are to log CAN bus data and to display information to the driver. To do this, the device requires input from the vehicle's 2 CAN busses and power from the vehicle's low voltage systems. In addition to this, the device must allow the driver to interact with the GUI to change what is displayed, requiring an input such as touchscreen. Lastly, the device must allow operators of the vehicle to pull logged data off of the device, or to push updates to the device. This is satisfied by the addition of an Ethernet interface, adding the logged data output and firmware input. These inputs and outputs are summarized in both *Figure 6* and *Table 7*.

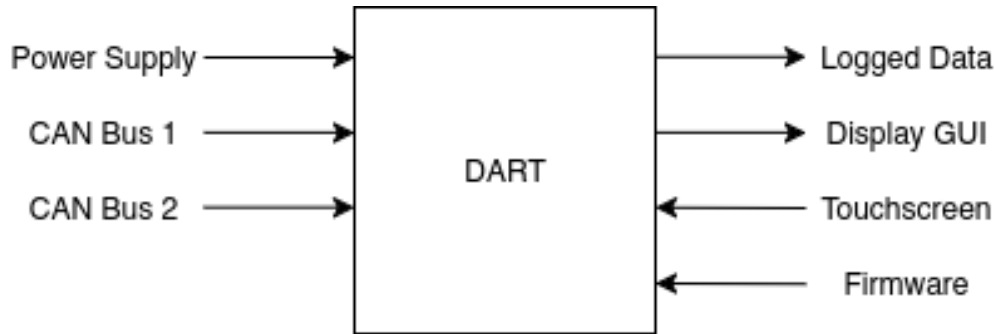


Figure 6: Level 0 Functional Block Diagram

Table 7: Level 0 DART Block Input / Output Table

Module	DART
Designer	Cole Barach, Ethan Zifzal, Matt Sullivan, Jacob Koshel
Inputs	Power Supply [11, 28] VDC, First CAN 2.0B Bus, Second CAN 2.0B Bus, Touchscreen controls, Firmware binaries (for programming)
Outputs	Logged Data (MDF file format), Display GUI
Functionality	Device displays CAN signals in real-time, Logs CAN data internally for later analysis.

In order to implement the *Level 0* diagram, a CPU-oriented architecture was designed. In this architecture, the SoM (referred here as Compute Module) acts as the device's central processor, being the platform on which all software is developed. Supporting the SoM are the interfaces for each of the device's peripherals, being the CAN Interfaces, Storage, Display, and

Magnetics. The central processor must also be fed a regulated power input; this regulation is performed by the power input stage. This architecture is laid out in *Figure 7*.

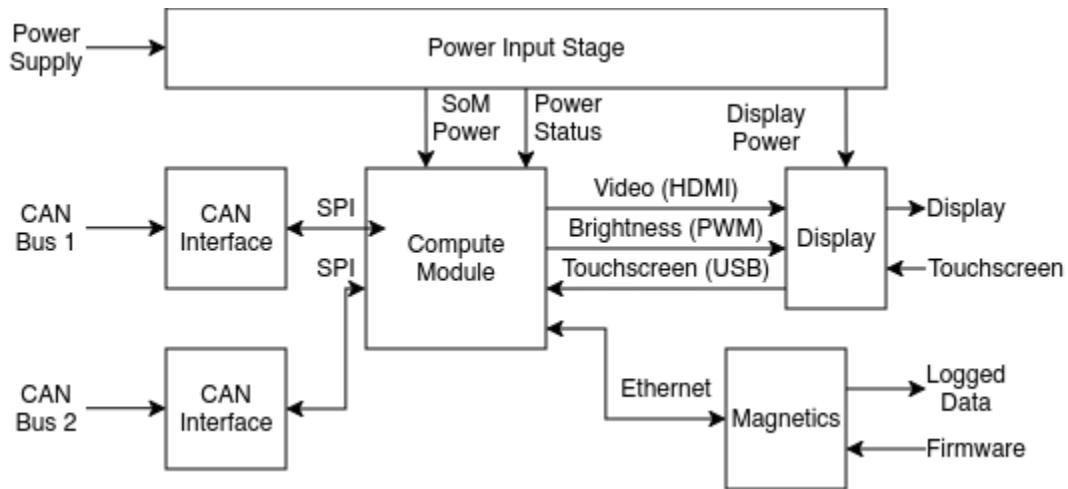


Figure 7: Level 1 Functional Block Diagram

The level 1 function block diagram was implemented in Altium Designer as seen in *Figure 8*, with each schematic block corresponding to a block in the diagram. Each schematic block has a sheet associated with it, where the inputs and outputs are passed through. This hierarchical scheme was used to better abstract the design of the device by making it more comparable to a block diagram.

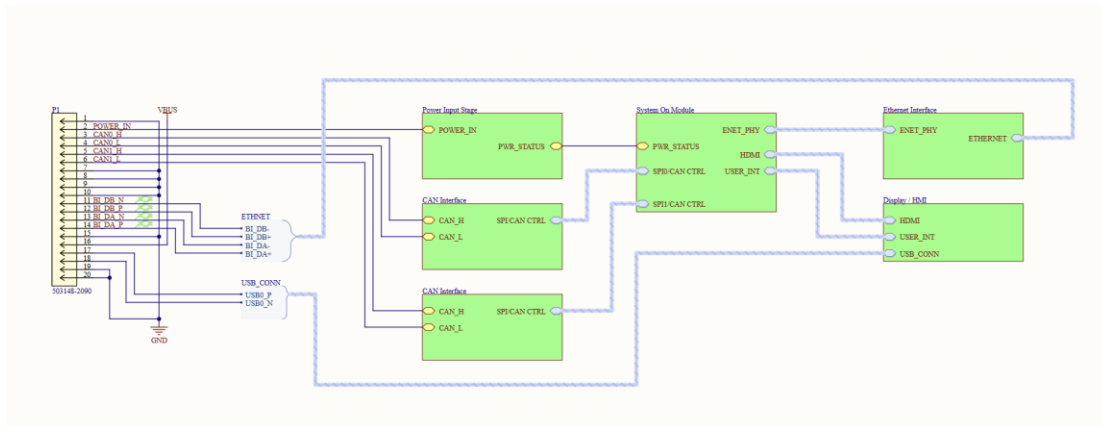


Figure 8: Altium Implementation of the Level 1 Functional Block Diagram

The SoM, again acting as the central processor, is responsible for carrying out the responsibilities of all the project's software. While these responsibilities are elaborated on further

in the *Software Design* section, they can be briefly summarized as logging CAN bus data and rendering the GUI. The hardware of the SoM defines the inputs and outputs made available, which are reduced down to the subset that is required for this project. These responsibilities, inputs, and outputs are summarized in *Table 8*.

Table 8: Level 1 Compute Module Input / Output Table

Module	Compute Module (SoM)
Designer	Ethan Zifzal
Inputs	SoM Power, Power Status, Comms, Filesystem, Ethernet, Touchscreen
Outputs	Comms, Video, Brightness
Functionality	Reads and decodes CAN messages from the 2 CAN busses, Displays specified CAN signals on a GUI, Logs all received CAN messages to a filesystem, Exposes logged data through ethernet for later access

Figure 9 shows the Compute Module interface in the schematic. To make the schematic more readable, the Compute Module was split into two blocks representing each of the 100 pin connectors. Net harnesses were used in Altium to conveniently connect multiple signals within a single harness to improve the modularity and legibility of the schematic. Figure 8 demonstrates the usefulness of this feature as the blue lines connecting each block together are a collection of nets under the umbrella of a single protocol or definition. These are indicated by blue squares with red text in Figure 8. Additionally, the system-on-module block was where the differential pair net directives were defined. Since the net harness passes the signal data to other sheets of the schematic, the differential directives only need to be applied on this sheet.

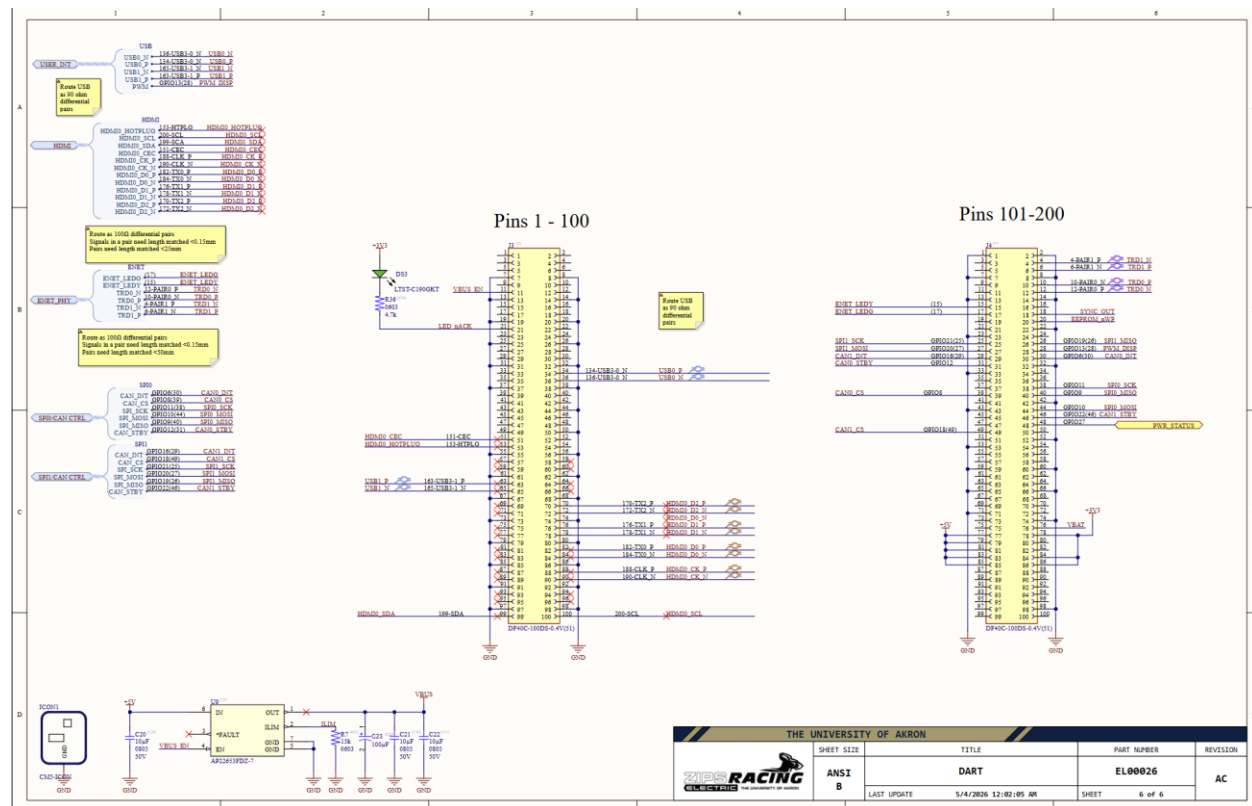


Figure 9: Altium Implementation of the Level 1 System-on-Module

The power input stage is responsible for regulating the power supplied to the SoM and display of the device. Most importantly, the power input stage must provide the SoM with an early notification of the device powering down. This allows the SoM to gracefully terminate the data log it is writing to, as doing so ungracefully introduces the risk of file corruption and data loss. To achieve this, the power input stage must detect when the device's supply power has been cut and notify the SoM while powering it off the stored energy from the bulk capacitance. This internal energy storage must be capable of powering the SoM long enough for it to gracefully terminate, as discussed in the *Design Calculations* section. The *power status* output is the signal used to notify the SoM. In addition to this, the power input stage must provide the more common responsibilities of a power regulator, mainly voltage regulation. The inputs, outputs, and responsibilities of the power input stage are summarized in *Table 9*.

Table 9: Level 1 Power Input Stage Input / Output Table

Module	Power Input Stage
Designer	Matt Sullivan
Inputs	Power Supply (11 - 28 VDC)
Outputs	SoM Power, Power Status, Display Status
Functionality	Voltage regulation, Power supply hold-up, Early power supply shutdown notification

The *Power Input Stage* has been further decomposed into the diagram shown in *Figure 10*. In this, the input power is fed through a series of circuits that regulate and hold up the supply voltage. The first component, the *Capacitance Bank*, provides the energy storage for holding the SoM power on during the shutdown sequence consisting of a combination of electrolytic and ceramic capacitors chosen to improve transient performance. The calculations for this energy storage are discussed in the *Design Calculations* section. The next circuit, *Power Monitoring*, implements the shutdown notification, specifically, the input power is compared with an internal reference to detect when it has gone below a specific voltage. The falling of the input voltage is interpreted as the device being powered down, so this signal is fed out as the *Power Status* signal. The last circuit, *Power Regulation*, implements a 24V to 5V voltage regulator for stepping the input voltage down to a level the SoM and display can operate at. *Table 11*

summarizes these responsibilities. The responsibilities of these circuits are summarized in *Tables 10, 11, and 12*. The implementation of these circuits is shown in *Figure 11*.

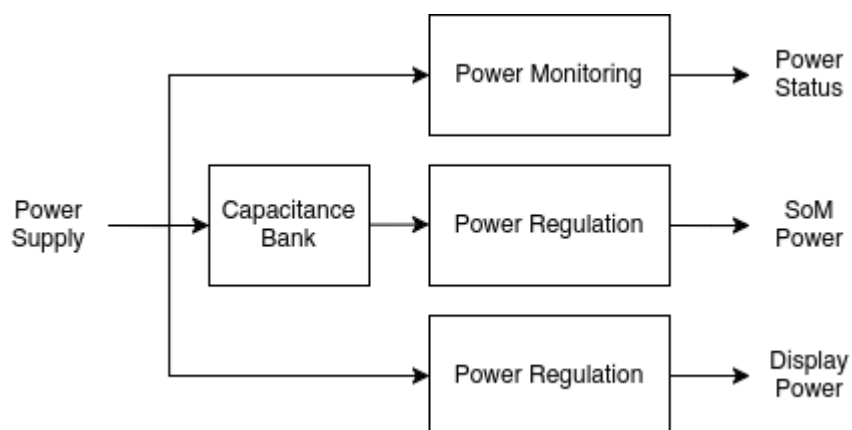


Figure 10: Level 2 Power Input Stage Functional Block Diagram

Table 10: Level 2 Power Regulation Input / Output Table

Module	Power Regulation
Designer	Matthew Sullivan
Inputs	Power Supply
Outputs	Regulated Voltage
Functionality	Converts 11-28V variable input power to a regulated DC output voltage of 5V

Table 11: Level 2 Power Monitoring Input / Output Table

Module	Power Monitoring
Designer	Matthew Sullivan
Inputs	Supply Power
Outputs	Power Status
Functionality	Monitors input voltage for detecting system shutdown, Indicates system shutdown to SoM

Table 12: Level 2 Capacitance Bank Input / Output Table

Module	Capacitance Bank
Designer	Matthew Sullivan
Inputs	Supply Voltage
Outputs	Capacitor Held-up Voltage
Functionality	Bulk capacitance storage to permit the SoM the enough time to safely power off.

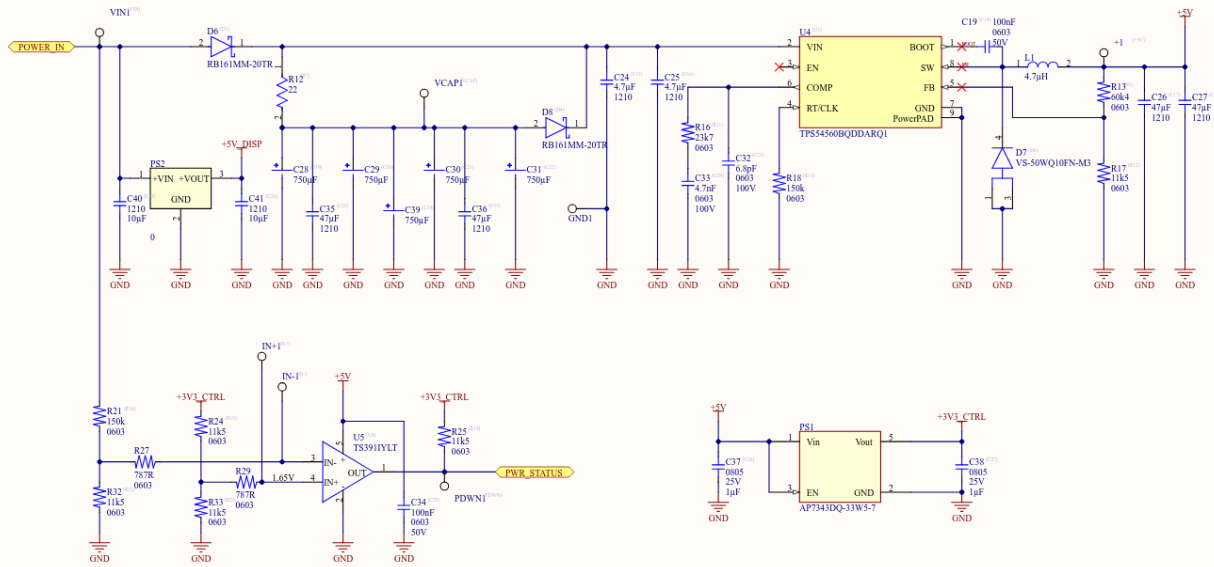


Figure 11: Altium Implementation of the Power Input Stage

In the Magnetics block of *Table 13*, Bob Smith termination was used to terminate the center taps of the pulse transformer as seen in *Figure 12*. The purpose of this termination is to provide an impedance-matched path back to the ground of the device. The circuit involves a series resistor and capacitor connected to each pair. Then all the pairs are connected in series to a single capacitor with a high voltage rating to block DC and low frequency ground loops and protect the device under potential EMI spikes.

Table 13: Level 1 Magnetics Input / Output Table

Module	Magnetics
Designer	Ethan Zifzal
Inputs	Ethernet, Firmware
Outputs	Ethernet, Logged Data
Functionality	Ethernet PHY isolates compute module from ethernet with pulse transformer utilizing a Bob Smith Termination Circuit

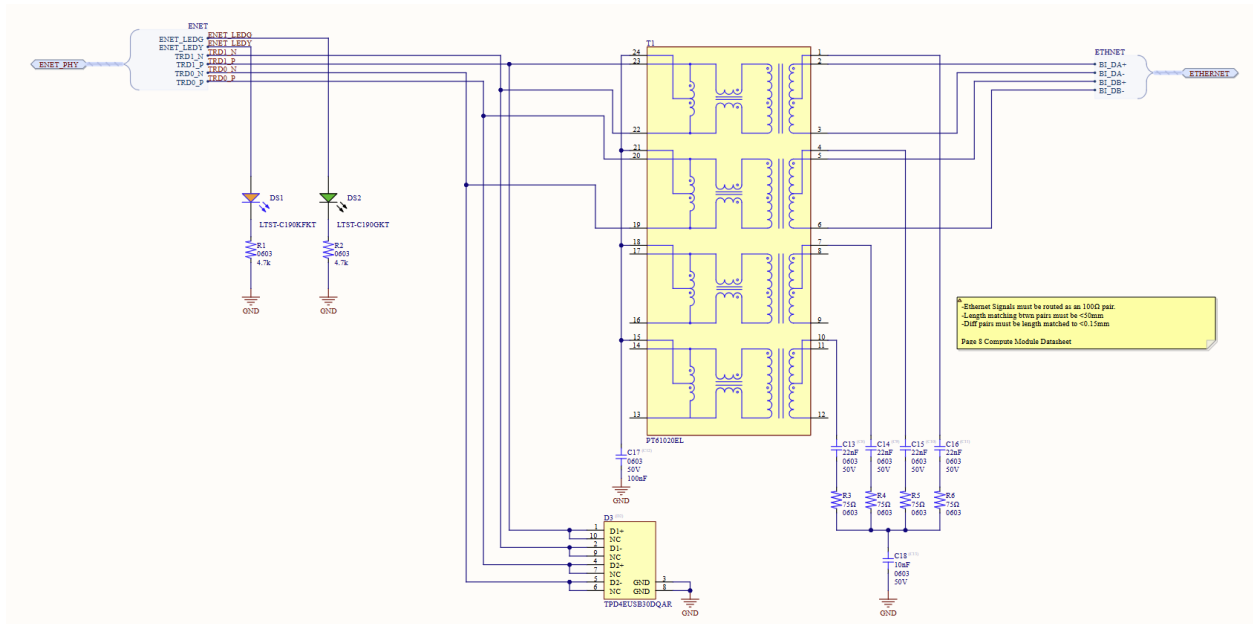


Figure 12: Altium Implementation of the Level 1 Magnetics Block

The display interface uses HDMI for video transmission. While a simpler protocol could have been used to reduce cost and complexity, there is a much larger market for HDMI displays. Since the device will be used outdoors, it was necessary to use an HDMI display as there are far more outdoor display options that use HDMI. Additionally, the display used in this project interacts with the SoM through USB for touch screen control, which allows the driver to quickly interact with the device. The realization of this circuit is provided in *Figure 13*.

Table 14: Level 1 Display Input / Output Table

Module	Display
Designer	COTS
Inputs	Display Power, Video, Brightness, Touchscreen
Outputs	Display, Touchscreen
Functionality	Displays video from the SoM to the driver, Brightness controlled by SoM, Relays touchscreen input to the SoM

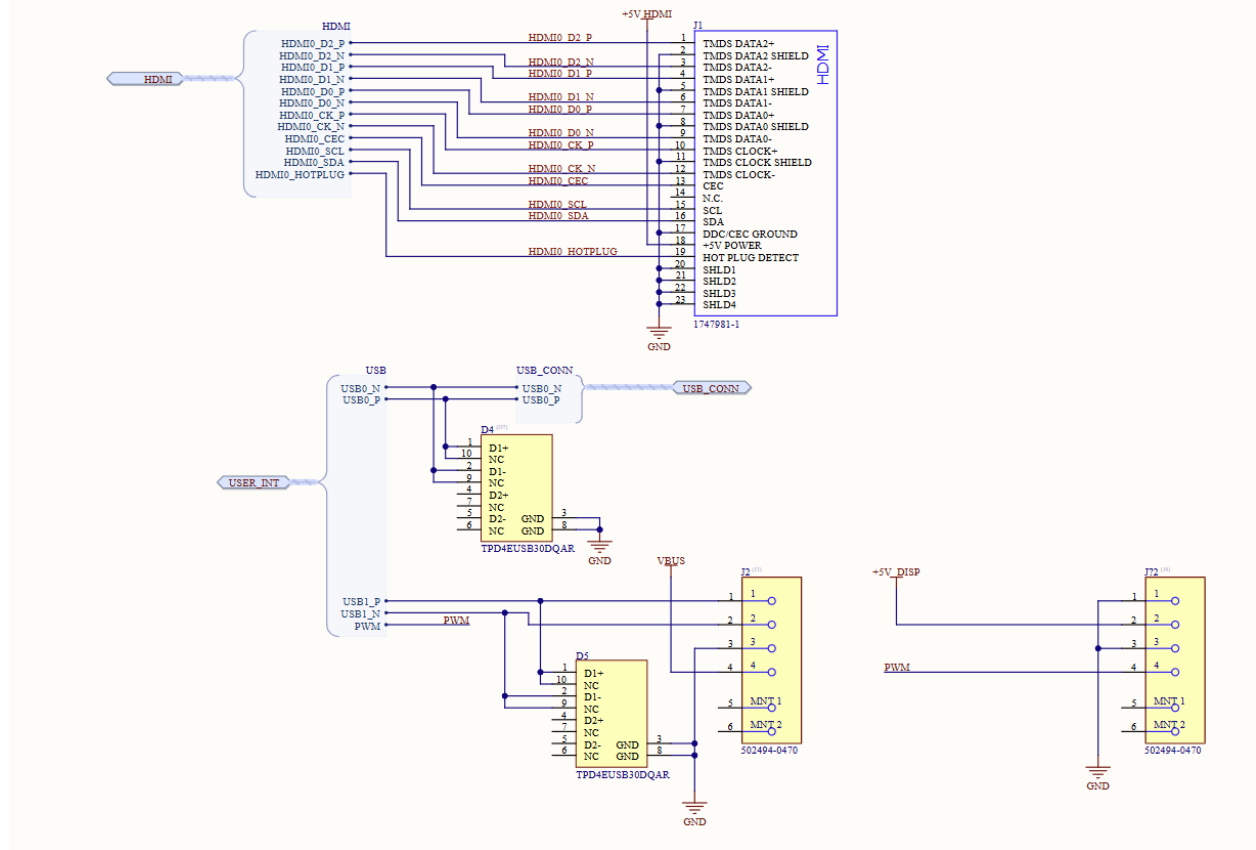


Figure 13: Altium Implementation of the Level 1 Display Block

The CAN controller and CAN transceiver implement the CAN block of *Figure 14*. As mentioned in the hardware architecture section, a combined integrated circuit was selected to minimize the physical size of this circuit seen in *Figure 15*. This integrated circuit is responsible for bridging communication between the differential CAN signals to digital CAN signals and finally to SPI signals as the SoM does not have a built-in CAN controller. Furthermore, ESD protection was added in the form of two TVS diodes in a single integrated circuit meant specifically for CAN bus implementations.

Table 15: Level 2 CAN Interface Input / Output Table

Module	CAN Interface
Designer	Ethan Zifzal
Inputs	CAN bus messages, SoM communications
Outputs	SoM communications

Functionality	Implements CAN bus negotiations (data link layer) and physical layer logic level translation.
---------------	---

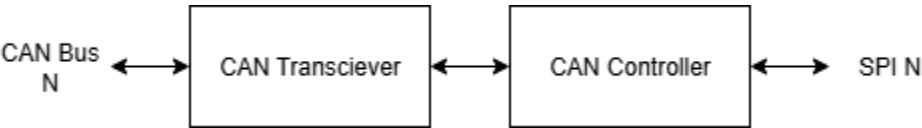


Figure 14: Level 2 CAN Interface Functional Block Diagram

Table 16: Level 2 CAN Transceiver Input / Output Table

Module	CAN Transceiver
Designer	Ethan Zifzal
Inputs	CAN Bus, TTL CAN
Outputs	CAN Bus, TTL CAN
Functionality	Performs CAN physical layer logic level translation.

Table 17: Level 2 CAN Controller Input / Output Table

Module	CAN Controller
Designer	Ethan Zifzal
Inputs	TTL CAN, SPI Bus
Outputs	TTL CAN, SPI Bus
Functionality	Performs CAN data-link layer arbitration

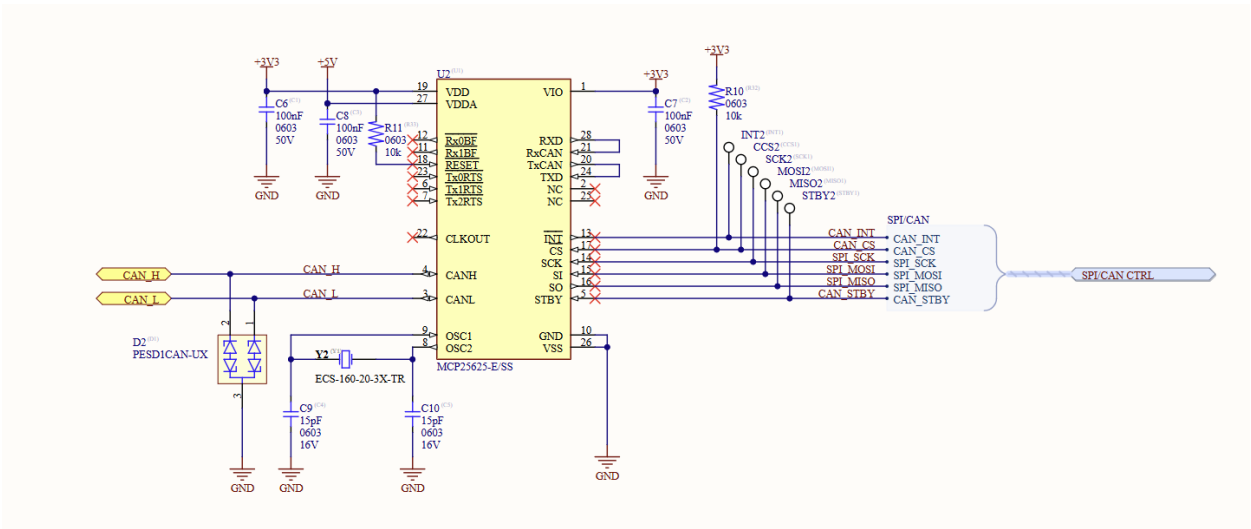


Figure 15: Altium Implementation of the Level 2 CAN Controller and CAN Transceiver Blocks

Upon the completion of the schematic, the PCB layout was started. The PCB dimensions were chosen to follow the dimensions of the display. The manufacturer of the display,

Newhaven, provides CAD files for their display. Their display has a circuit board connected to the LCD with mounting holes. This solid body was extracted from their CAD model and used as the basis for the PCB design. Then the custom PCB was designed around these dimensions to create a board stack which allowed for denser packaging of the device.

Routing of the PCB was primarily constrained by connector placement on the display and the area taken by the SoM. The connector layout of the PCB needs to mirror that of the display to use a board-to-board connector for the HDMI interface. Once the connector constraints were accounted for, the SoM was placed roughly in the center of the board. This placement permits enough space to be allocated for the bulk capacitors needed for pre-charge. Next, the high frequency signaling was routed for Ethernet, HDMI, and USB. As mentioned in the hardware architecture section, impedance-matched differential pairs were used to connect the high frequency circuitry. Finally, the power and low frequency embedded circuits were routed with care being taken to avoid placement near the high frequency lines. The routing of the circuit board is displayed in *Figure 16* and the physical PCB itself is shown in *Figure 17*. An example of the high frequency routing is easily visible on the right side of the board, where the HDMI pairs are routed to the connector on the top right of the board from the SoM's second 200 pin connector.

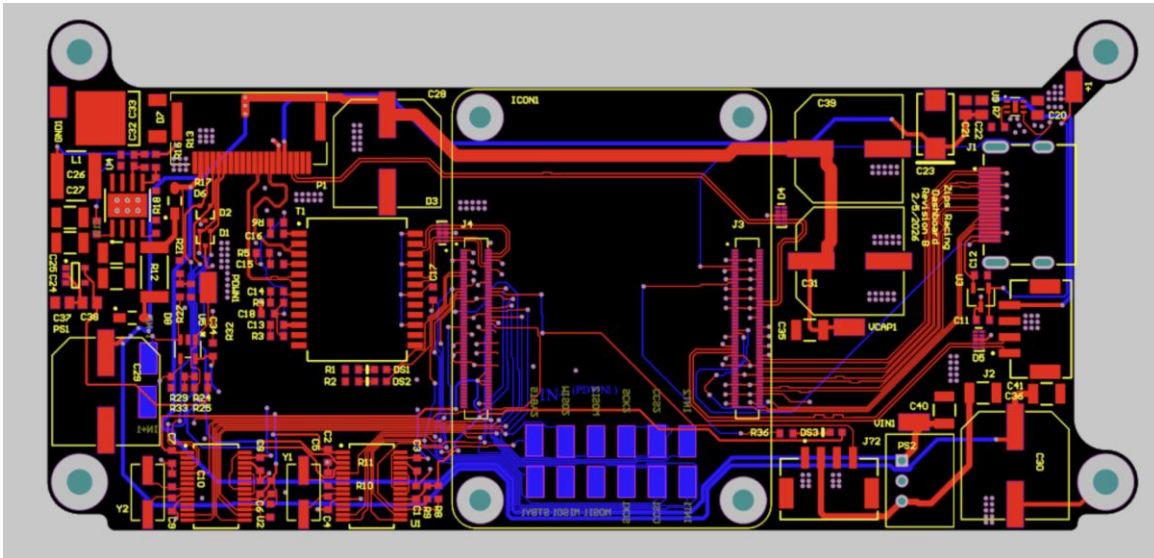


Figure 16: Latest Device PCB Layout

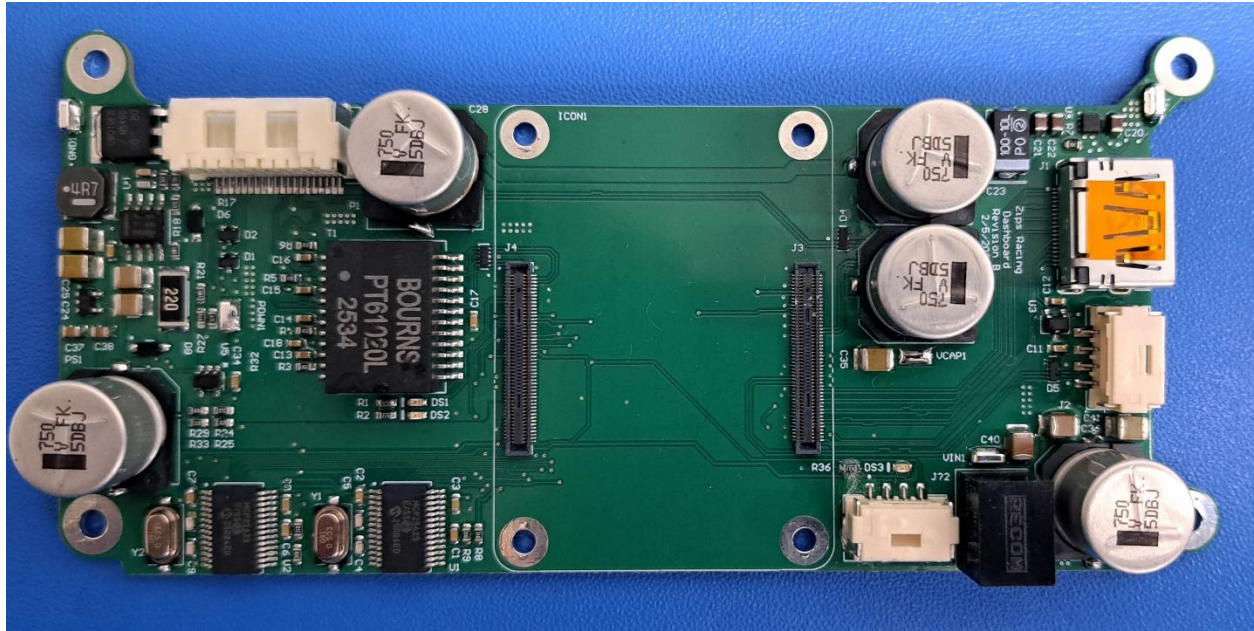


Figure 17: Latest Revision of the Device's PCB.

5.2 Software Design (CB):

The responsibilities of the device's software, as discussed in the *Engineering Analysis* section, will be distributed across 3 core applications: the graphical user interface (GUI), the data logging application, and the init-system. In order to improve the software's flexibility and allow reuse of code, the GUI and data logging application are dependent on both custom and 3rd party libraries. While the applications are designed to be independent, a small degree of interaction is required between them. The init-system will use operating system signals to communicate with the GUI and data logger. The data logger and the GUI will use CAN bus to communicate with one another. *Figure 18 details a high-level view of each application and their dependencies.* Note that kernel modules and trivial dependencies are omitted for simplicity. All software excluding the Linux kernel represent dependencies via each modules' application binary interfaces (ABI). Specifically, the ABI exposed by each module is the set of functions and global variables that are defined with external linkage.

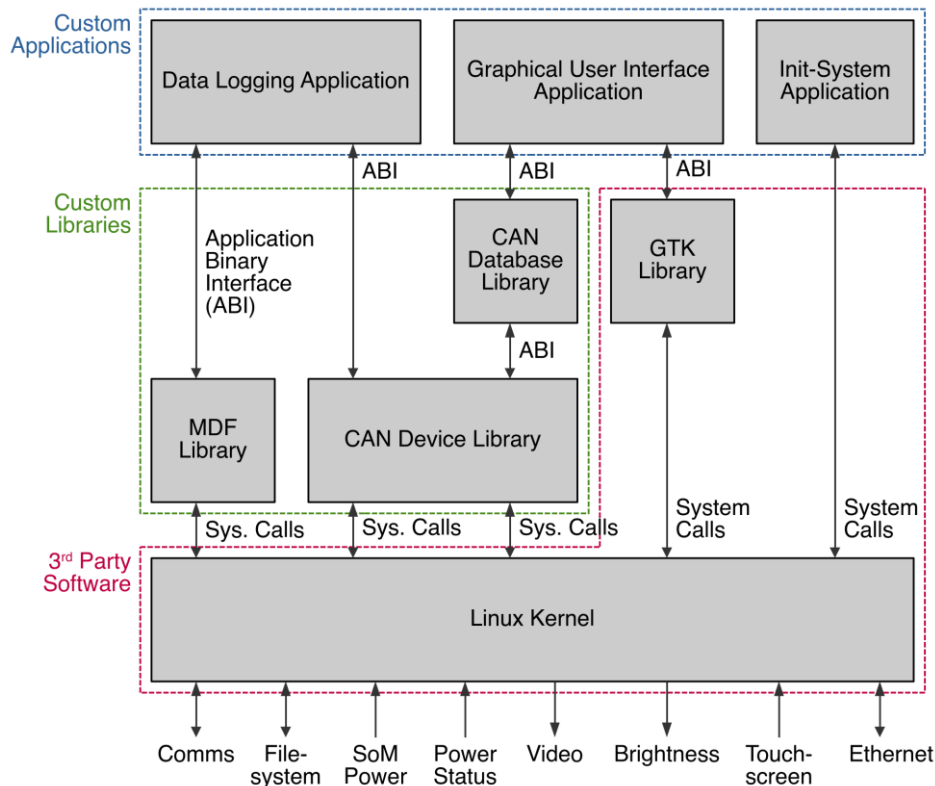


Figure 18: Level 2 Compute Module Functional Block Diagram

The Linux kernel is the foundation of the device’s software. In *Figure 18*, the kernel represents the SoM’s hardware / software interface, as this is the primary function the kernel provides. All arrows entering the kernel from the top represent the software interface, as defined by the POSIX specification. All arrows entering from the bottom represent the hardware interface, as defined by the SoM’s datasheet and summarized in *Table 8*. These inputs, outputs, and responsibilities are summarized in *Table 18*.

Table 18: Level 2 Operating System Input / Output Table

Module Name	Operating System
Designer	3 rd Party
Inputs	Comms, Filesystem, Touchscreen, Ethernet, Power Status
Outputs	Comms, Filesystem, Video, Brightness, Ethernet
Functionality	Performs hardware abstraction, provides programming concurrency, manages storage filesystem, manages RTC

The CAN device library is a custom library used for abstracting the device’s CAN interfaces. Specifically, this library provides a portable, object-oriented, and polymorphic interface for interacting with CAN buses. The main appeal of this is to make applications built on top of this interface independent of the operating system and CAN controller in use. With this, the applications can be tested in either Windows or Linux, and with either SocketCAN or SLCAN based hardware. This is an important step in improving the maintainability of the project, as maintainers of the project are likely to own Windows PCs, which are only compatible with SLCAN based hardware, which requires a different implementation to the device’s Linux environment which uses SocketCAN based hardware. A summary of these responsibilities is provided in *Table 19*. The implementation of this library is referenced in *Appendix B*.

Table 19: Level 2 CAN Device Library Input / Output Table

Module Name	CAN Device Library
Designer	Cole Barach
Inputs / Outputs	System Calls, Application Binary Interface
Functionality	Abstracts access to CAN bus, Polymorphic interface for different hardware implementations, Encapsulation of OS-specific software, Object-oriented access to multiple CAN busses

The CAN database library is a custom library built on top of the CAN device library. The main goals of this library are to implement the CAN DBC standard and further abstract access to the CAN bus. Specifically, the CAN database library abstracts one or more CAN buses as a relational database of CAN signals. Constructed from the information parsed in a set of CAN DBC files, each signal is attributed a name, value, and validity. Normally, the value of each signal is the most recent value received from the CAN bus. If the message containing a signal's value has not been received for a certain period of time, the signal is timed out and the value is invalid. The name of each signal is the key used to identify it, therefore, if the bit packing, scale factor, ID, or any other quality of a signal is changed, applications built on top of the CAN database are unaffected so long as the DBC file is updated as well. This is a major step in being able to port the device to future vehicles, as, when messages on the CAN bus are changed, the only required maintenance is to update the DBC file with said changes. A summary of these responsibilities is provided in *Table 20*. The implementation of this library is referenced in *appendix B*.

Table 20: Level 2 CAN Database Library Input / Output Table

Module Name	CAN Database Library
Designer	Cole Barach
Inputs / Outputs	Application Binary Interface
Functionality	Abstracts access to CAN bus, Implementation of CAN DBC standard, Configuration based portability, Signal expiration and validation

The MDF library is a custom library implementing the ASAM MDF version 4 standard and the corollary bus logging specification. At its base, this library defines a set of utilities for reading and writing the linked binary blocks of information that all MDF files consist of. This also defines the contents and record IDs of specific MDF blocks. Built on top of this base is the implementation of the bus logging specification, which is a set of utilities for writing the header and records of a MDF CAN bus log file. A summary of these responsibilities is provided in *Table 21*. The implementation of this library is referenced in *Appendix B*.

Table 21: Level 2 MDF Library Input / Output Table

Module Name	MDF Library
Designer	Cole Barach
Inputs / Outputs	System Calls, Application Binary Interface
Functionality	Reading and writing linked blocks to MDF files, Implementation of the MDF bus logging specification

The GTK library is a 3rd party library used for creating GUIs. In this project specifically, it is being used to create the GUI application. GTK defines a framework for creating windows, the container for a GUI, and widgets, the contents of a GUI. GTK defines a core set of widgets that can be used to construct more complex elements. Some examples of the core widgets are labels, text boxes, buttons, and progress bars. Applications using GTK can create custom bindings for the information displayed by a widget and the source of the information. GTK also provides an event loop that runs at a set frequency to manually update elements of the GUI. These responsibilities are summarized in *Table 22*.

Table 22: Level 2 GTK Input / Output Table

Module Name	GTK
Designer	3 rd Party
Inputs / Outputs	System Calls, Application Binary Interface
Functionality	Creation of GUI framework, Definition of core widgets, Implementation of event loop

5.2.1 The Graphical User Interface

The GUI is responsible for rendering the information to be displayed to the driver. Specifically, the GUI satisfies engineering requirements 2 & 3. Due to the large volume of information and the small area of the device's display, the GUI must be selective with its design. One simple way to improve this is to paginate the GUI, as there is often only a small subset of information that is relevant at a given time. An intuitive way of dividing these pages is to do one per event at the competition, plus pages for debugging and diagnostics for when the vehicle is stationary. The main pages the GUI has been divided into are the autocross page, the endurance page, the debugging page, and the battery management system page. A summary of these responsibilities is provided in *Table 23*.

Table 23: Level 2 Graphical User Interface Application Input / Output Table

Module Name	Graphical User Interface Application
Designer	Jacob Koshel
Inputs / Outputs	Application Binary Interface
Functionality	Visually displays specific CAN signals in real-time, Pages for different events

The autocross page is intended for use in the autocross and skidpad events. In these events, speed is the most important factor, hence it is given the most focus on the GUI. The current version of this page is shown in *Figure 19*. In the top left of the page, the data logger status and session number are displayed. This information tells the driver that the data logging application is running and what session number it is currently logging into. This is achieved by having the data logging application regularly transmit a status message onto the vehicle's CAN busses. The status of the vehicle's powertrain is summarized to the driver on the right side panel. This information is less immediately relevant, so it is smaller and off to the side. Lastly an array of buttons at the bottom of the display allow the driver to navigate to the other pages.

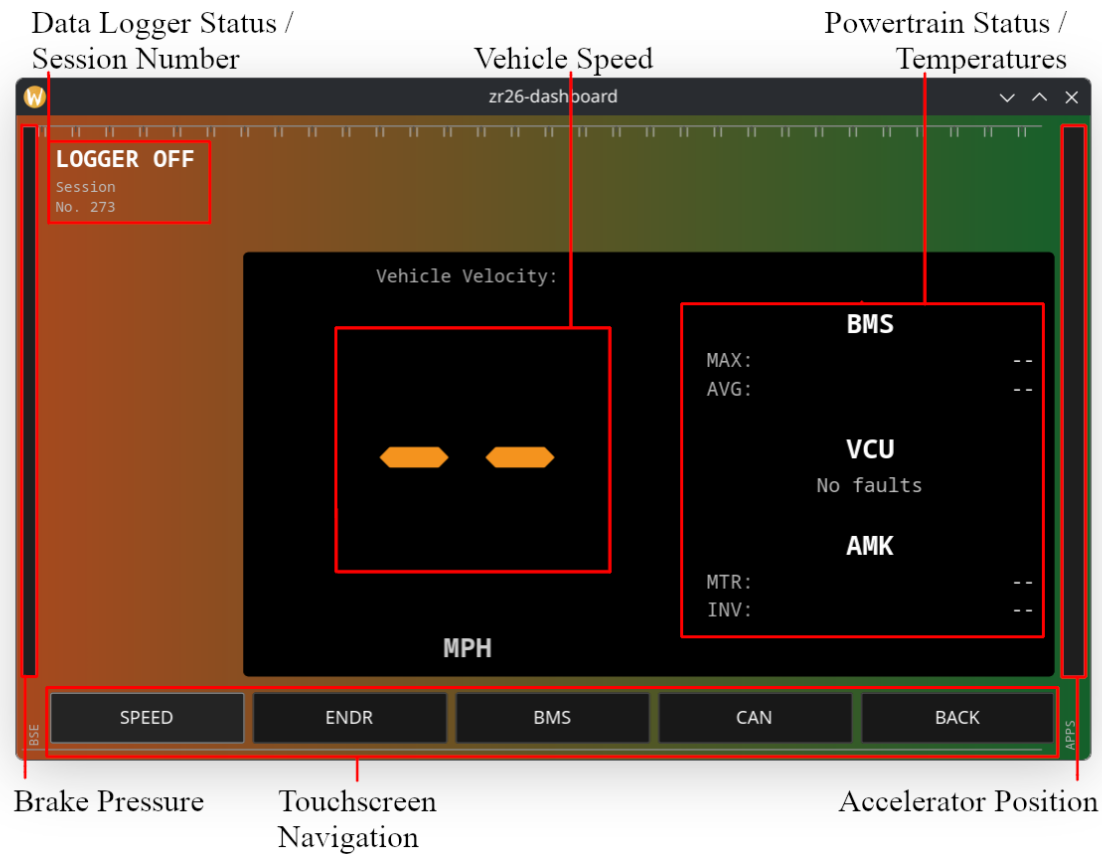


Figure 19: Current version of the autocross page. Red annotations highlight specific portions of the GUI. These are not part of the design.

The endurance page is intended for use in the endurance event. In this event, the battery capacity and powertrain temperatures are the most important factors for the driver to pay attention to. The current implementation of this page is shown in *Figure 20*. The layout is very similar to that of the autocross page, with specific signals swapped out for more relevant ones. Specifically, the vehicle speed is replaced with the voltage of the high voltage battery, the instantaneous power output of said battery, and the total energy delivered from said battery.

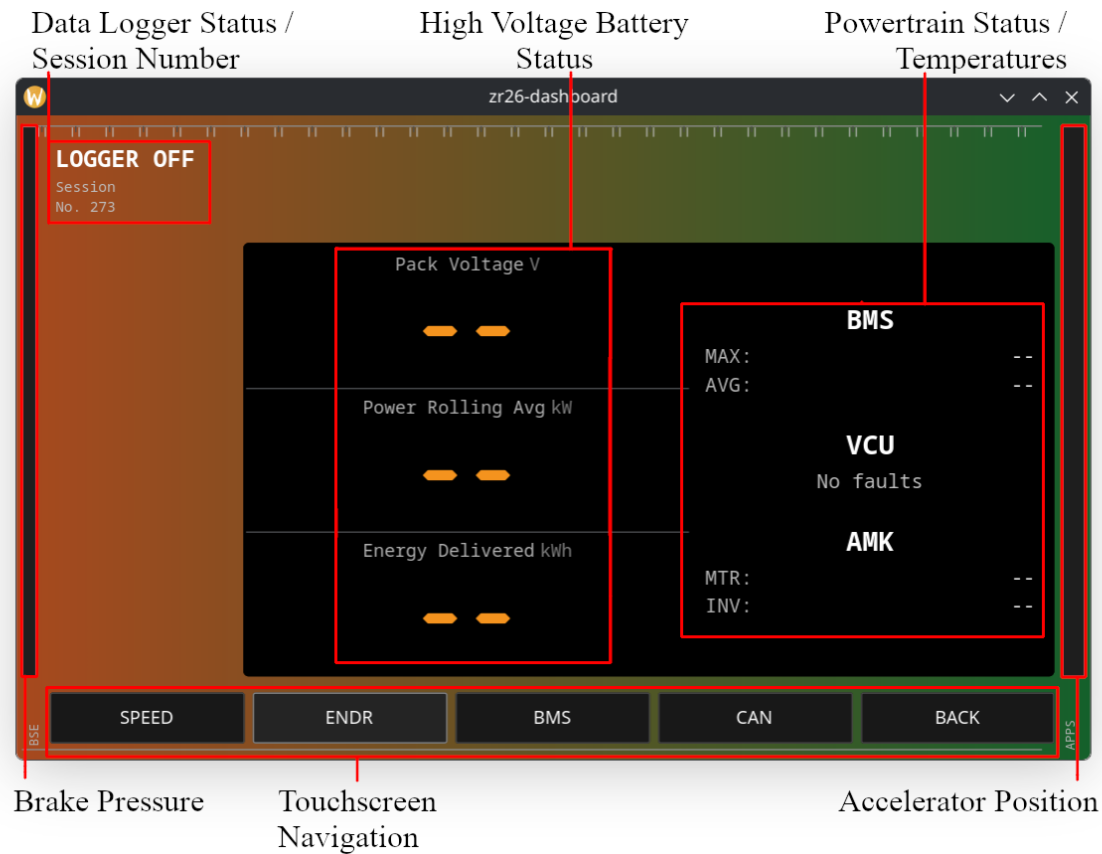


Figure 20: Current version of the endurance page. Red annotations highlight specific portions of the GUI. These are not part of the design.

The battery management system page is used for monitoring the battery management system (BMS) of the vehicle's accumulator. The BMS reports voltages, temperatures, and fault conditions of all the cells and sense lines in the accumulator. This information is important for estimating both the state of charge and state of health of the accumulator, as well as diagnosing fault conditions. While it is not necessarily required for this information to be present on the dashboard, it is incredibly beneficial in improving the usability of the vehicle. The current version of this page is shown in *Figure 21*. Currently, only cell voltages are reported, as it is still not clear on the best method of presenting the large volume of information. This page is not intended to be used while driving, therefore the contents can be made quite dense compared with the other pages.



Figure 21: Current version of the BMS page. Red annotations highlight specific portions of the GUI. These are not part of the design.

The debugging page, similar to the BMS page, is not intended to be used while driving. Its main purpose is to be used for debugging the vehicle’s other electrical systems. This is done by providing a comprehensive list of all the information available to the device, specifically, the value of every CAN bus signal. Each signal has its name, value, and unit displayed, as well as being grouped with the CAN message it belongs to. Due to there being hundreds of different signals, this page uses touchscreen scrolling.



Figure 22: Current version of the debugging page. Red annotations highlight specific portions of the GUI. These are not part of the design.

The C implementation of the GUI, due to its length, has been omitted from this report. The latest version can, however, be accessed via *Appendix B*.

5.2.2 The Data Logging Application

The data logging application is responsible for logging all information received from the vehicle's CAN busses. Specifically, this application covers engineering requirements 9 & 10. For every power cycle of the vehicle, the data logger must create a unique MDF file for all information logged in the session. For every CAN frame received in a logging session, the data logger must write the contents of said frame, along with a timestamp, into said MDF file. Upon termination of the application, as signaled by the init-system, the data logger must gracefully terminate the MDF file and close the file handler. Without this, the log is susceptible to corruption and data loss. Lastly, the data logger periodically transmits a status message to both of the vehicle's CAN busses, containing the approximate load of each CAN bus, the session number of the current logging session, and the number of error frames received since the last status message. This status message, in addition to being logged, is interpreted by the GUI to communicate said information to the driver. A summary of these responsibilities is provided in *Table 24*.

Table 24: Level 2 Data Logging Application Input / Output Table

Module Name	Data Logging Application
Designer	Cole Barach
Inputs / Outputs	Application Binary Interface
Functionality	Logs all received CAN messages to a unique MDF file, Transmission of periodic status message, Graceful termination of log file

A flow chart detailing the implementation of these responsibilities is provided in *Figure 23*. The entrypoint of the application, shown in the top right, starts both the initialization of the application's CAN interfaces and the creation of the session's MDF file. After this initialization, the application splits into two threads, one per CAN interface. Each thread sleeps until a CAN frame has been received, upon which a timestamp is generated and the type of frame is identified. After identification, the thread acquires exclusive access to the MDF file, where the contents, type, and timestamp of the frame are recorded. If the deadline for transmitting the status message has expired, the status message will be computed and transmitted. After transmission, the message is written into the data log. Lastly, regardless of whether the status message was transmitted or not, the thread will check whether a termination signal has been sent.

If not, the thread will return to the sleeping state waiting for the next CAN frame. If the termination signal has been sent, the thread will terminate and join with the main thread. Upon all threads having been joined, the main thread will close the MDF file and terminate itself.

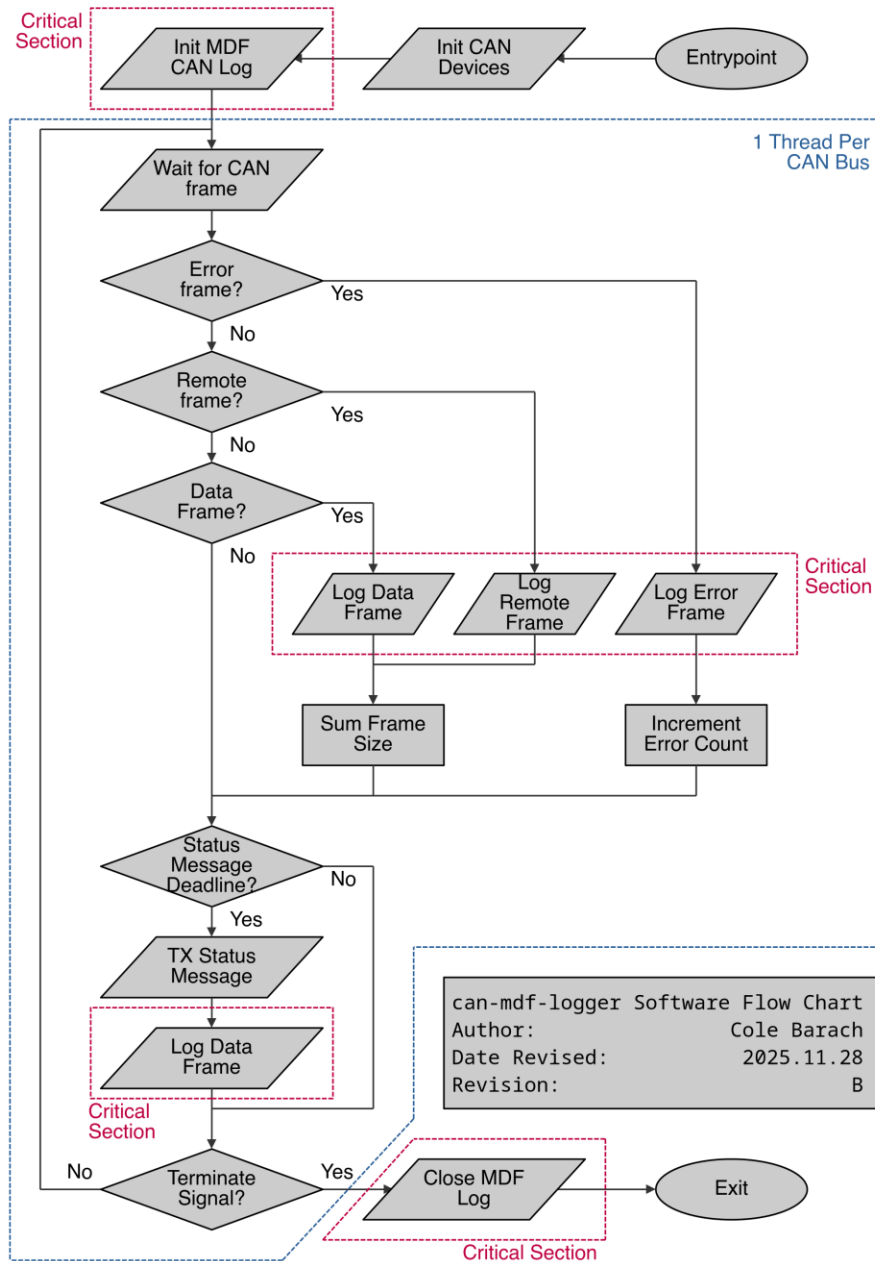


Figure 23: Data logging application software flowchart

The C implementation of this program, due to its length, has been omitted from this report, however, it may be accessed via *Appendix B*.

5.2.3 The Init System

The init-system is responsible for integrating both the GUI and the data logging applications. The main goal of the init-system is to launch said applications when the device boots and terminate said applications when the system is powering down. The device's power down sequence is signaled by the power input subsystem via hardware, hence the need to isolate this functionality in the init-system. The init-system is also responsible for bringing the CAN interface hardware online and configuring it for the correct baudrates. Even though multiple applications can share access to this hardware, only one application may perform this initialization. Because of this, the data logger and GUI are designed to expect this initialization be performed before they are launched.

It is important to note that, due to the large number of responsibilities it would require implementing, the init-system does not act as the init application of the operating system. Rather, Systemd is used as the init application and the init-system is launched as a system service. With this approach, the init-system is only responsible for managing the 1st party software discussed in this report and Systemd is only responsible for the 3rd party software composing the operating system.

Table 25: Level 2 Init-System Input / Output Table

Module Name	Init-system
Designer	Cole Barach
Inputs / Outputs	OS System Calls
Functionality	Launch the data logging and GUI applications on system power up, Monitors the power status for power loss events, Notifies applications of power loss and safely shuts system down

A flow chart detailing the implementation of the init-system is provided in *Figure 24*. The entrypoint of the application, shown near the top right, starts the initialization of the CAN hardware. After this, the data logger and GUI applications are launched, each given references to the newly initialized CAN interfaces. After this, the init-system enters the sleep state waiting for the power status interrupt. This interrupt is triggered by a digital input from the power input subsystem, specifically when the device's power supply has powered down. After validating this interrupt has occurred and was not a spurious wakeup, the init-system will begin the teardown

sequence. In this, the data logging application is gracefully terminated and the dashboard GUI is killed. After both applications have terminated, the init-system will lastly de-initialize the CAN hardware and halt the operating system. The C implementation of this program, due to its length, has been omitted from this report, however, it may be accessed via *Appendix B*

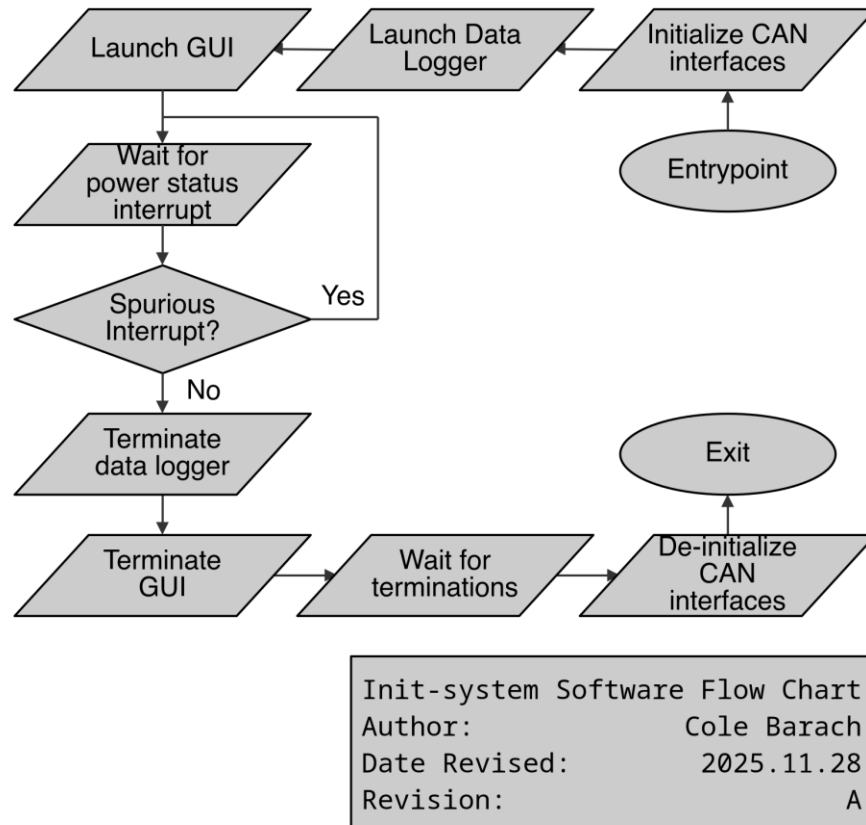


Figure 24: Init-system application software flowchart

5.3 Lessons Learned (CB, EZ)

On the software side, one of the things that ended up being more difficult than expected was gracefully shutting down the operating system. When drafting specifications for the power input stage, the minimum holdup time for the processor was chosen to 100 *ms*. If the processor does not complete its shutdown in this time, data logs or the operating system itself may become corrupted and unusable.

In the spring semester, when the software shutdown sequence was being tested, it was measured to be several seconds long. The largest contributor to this was found to be Systemd, the 3rd party init application that manages the operating system's services. To reduce this time within

specifications, each of the operating system's services had to be checked as to whether or not they required a graceful termination. If a service did not require this, it could be killed instead of terminated, removing its termination time from the shutdown sequence. The only service that was determined to require a graceful termination was the data logging application, as it is the only application that actively writes into the filesystem. With this change, the shutdown sequence was brought below the 100 *ms* threshold.

The design of the power input stage proved to be more difficult than initially anticipated. The first issue was the placement of capacitors on the 5 *V* rail to act as a hold up. The compute module was only functional from 5.25 *V* to 4.75 *V* meaning only a very slim amount of energy was usable and due to the energy stored in a capacitor exponentially increasing with voltage it was realized that charging these capacitors with the input voltage rail made more sense. Furthermore, another issue encountered was the type of capacitor used. Initially, supercapacitors meant for an RTC were used however these have a prominent downside being their high equivalent series resistance (ESR) especially during transients like power down which caused the SoM to brownout during power down. This was resolved by switching to electrolytic capacitors which had an ESR orders of magnitude lower than the previously used supercapacitors.

An unexpected mechanical issue was the reliability of the device's HDMI bridge. It was found after roughly a week of testing on Zips Racing's 2026 vehicle (ZR26) that the HDMI bridge can back out due to vibration. This causes the display to stop working, even when the device is powered and the data logger is running. For the current version of the device, this was fixed by adding a drop of hot glue between the back of the HDMI bridge and the enclosure wall. The elasticity of this adds a slight force holding the HDMI connectors in place. Since this fix, the issue has not reoccurred.

5.4 Financial Budget (EZ)

As the Zips Racing design team operates on a fixed budget our first requirement was that our device must be more affordable than the previously used MoTeC dashboard. The cost unit cost of our device was tabulated and found to be approximately \$340. Comparing the cost of this device to that of the MoTeC C125 at \$3500, the cost reduction is over 90%. However, this is not without tradeoffs, as the MoTeC system offers superior mechanical construction and a physically

smaller design. These tradeoffs have been determined to be acceptable primarily because of the custom solution's lower cost and Zips Racing's ability to improve it in the future.

Table 26: Per-unit Device Cost, in an Order Quantity of 5.

PCB Parts	\$53.73
Bare PCB	\$2.00
Autosport Connector	\$51.79
Display (NHD-5.0-HDMI-N-RTXL-CTU)	\$149.6
HDMI Bridge PCB (MCIB-HDMI/HDMI)	\$4.15
Mechanical Hardware	\$2.70
Wire	\$2.00
Internal Harness Connector	\$0.33
Internal Crimps	\$1.50
System on Module (SC1565)	\$80.00
Total Cost	\$337.80

6. Mechanical Sketch (EZ, CB):

Mechanically, the device consists of the display, the PCB, and the enclosure surrounding these two. The display, PCB, and SoM form a board stack. The PCB and SoM are bridged by two 100 pin connectors, and the PCB and display are bridged by HDMI, USB, PWM, and power. Mechanically, standoffs are used to secure both the SoM to the PCB and the PCB to the display. A CAD model of the SoM and PCB board stack is shown in *Figure 25*.

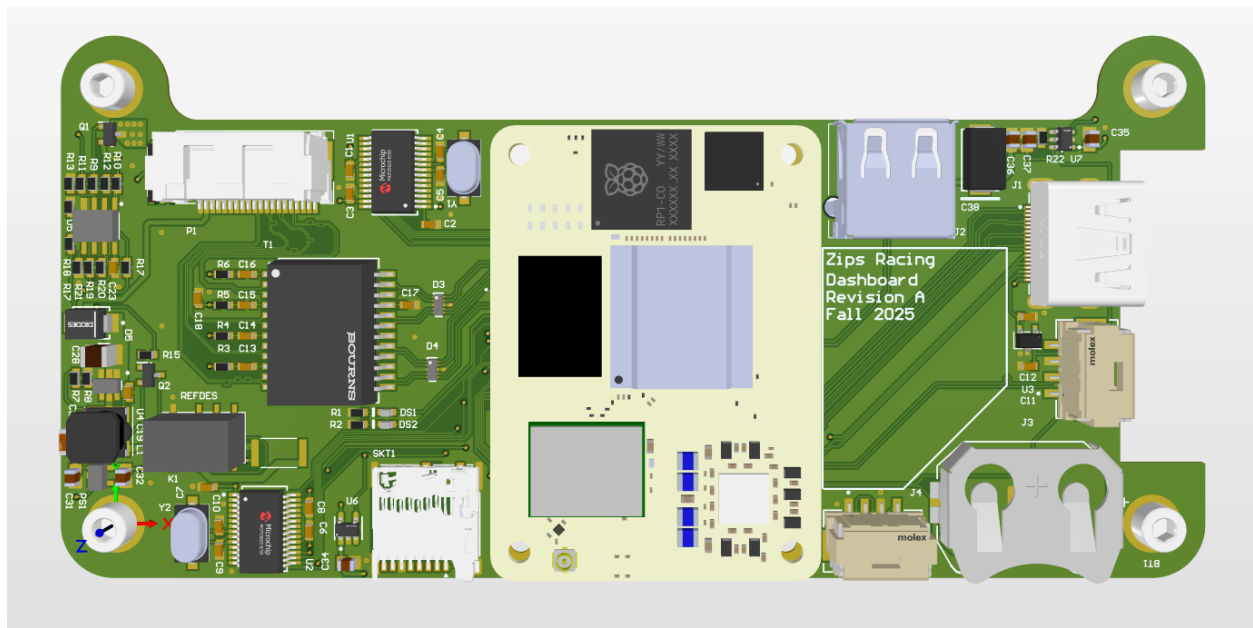


Figure 25: 3D CAD model of the custom PCB and the System-on-Module. To differentiate the two, the PCB is colored green and the SoM is colored tan.

The enclosure consists of two 3D printed parts using ABS filament and sealed with silicon. A front panel piece acts as a shell to connect the display and PCB together and a rear panel piece mounts to the front panel containing the device's connector to the vehicle. Four 40 mm M3 bolts cross the entire device seen in the exploded view which mounts to the vehicle and are then secured with nyloc nuts. The front, side, and exploded perspective of the enclosure design are shown in *Figures 26, 27, & 28*, respectively.

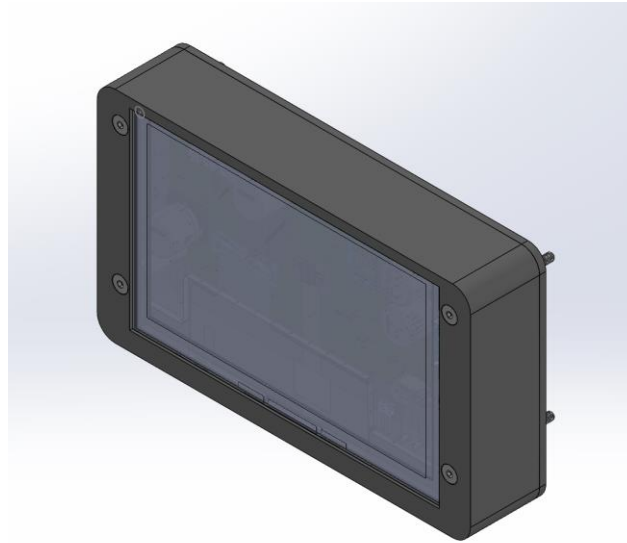


Figure 26: Enclosure Side Perspective

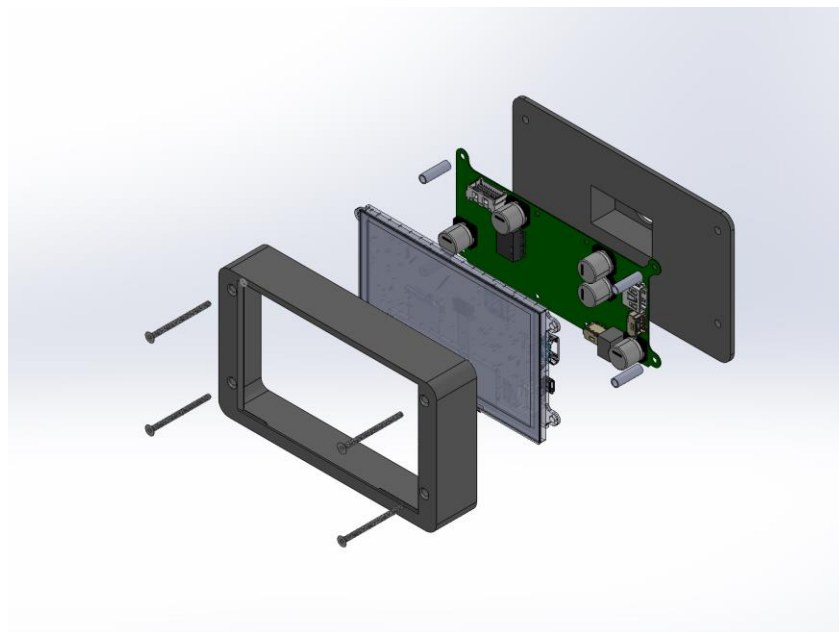


Figure 27: Enclosure Side Perspective Exploded View



Figure 28: Enclosure Front Perspective

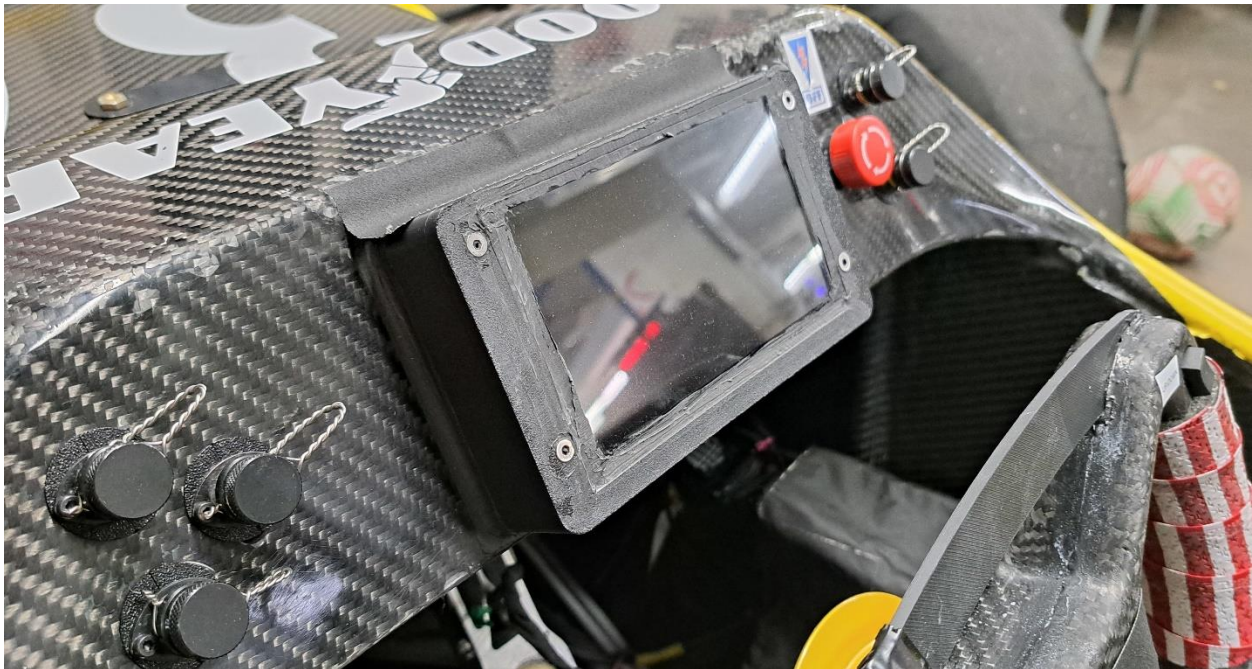


Figure 29: Final device design installed on the dashboard of ZR26. Rubber gromets were used to isolate the device from vibration and shock. Black silicon was used to seal the device from water.

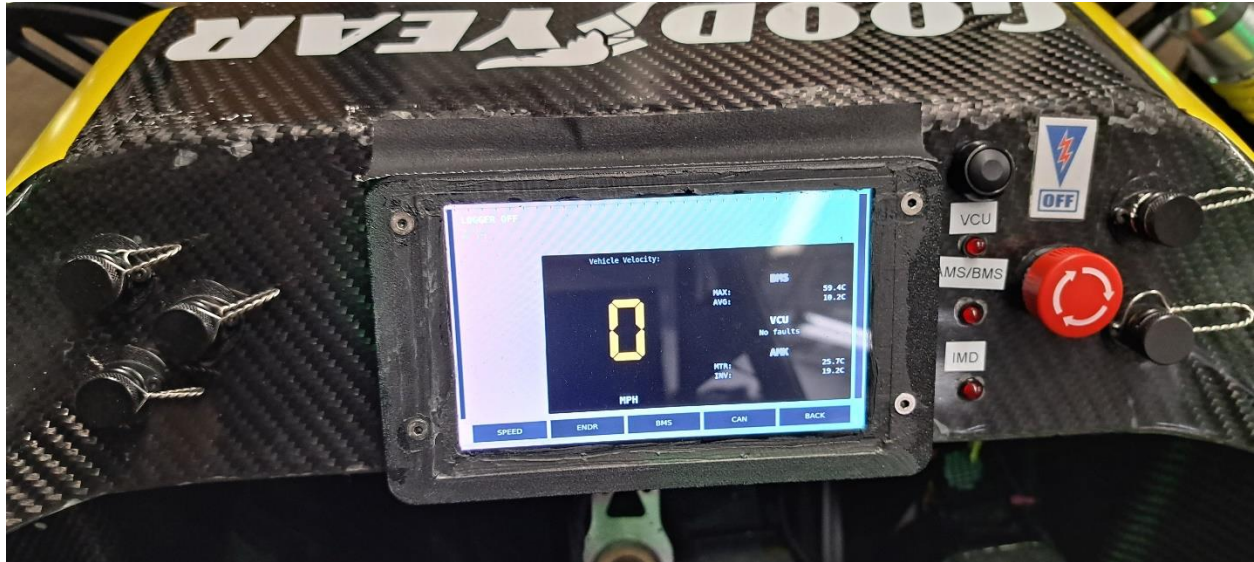


Figure 30: Final device design installed on ZR26 with power on. The autocross page is currently selected.

7. Team Information

Cole Barach, Computer Engineer
Jacob Koshel, Computer Engineer
Matthew Sullivan, Electrical Engineer
Ethan Zifzal, Electrical Engineer

8. Project Schedules (CB, EZ, JK, MS)

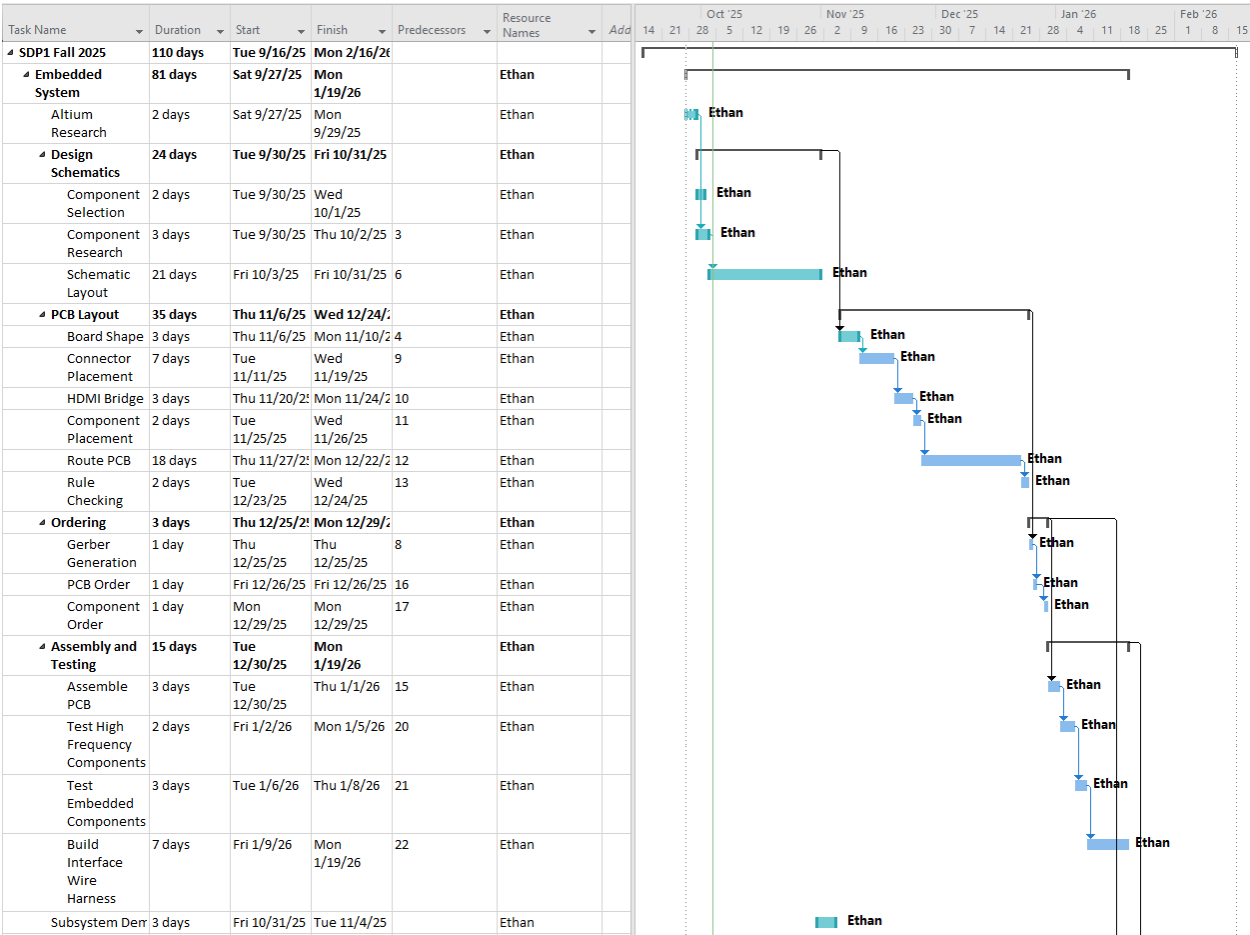


Figure 31: Embedded Systems Subsystem Gantt Chart

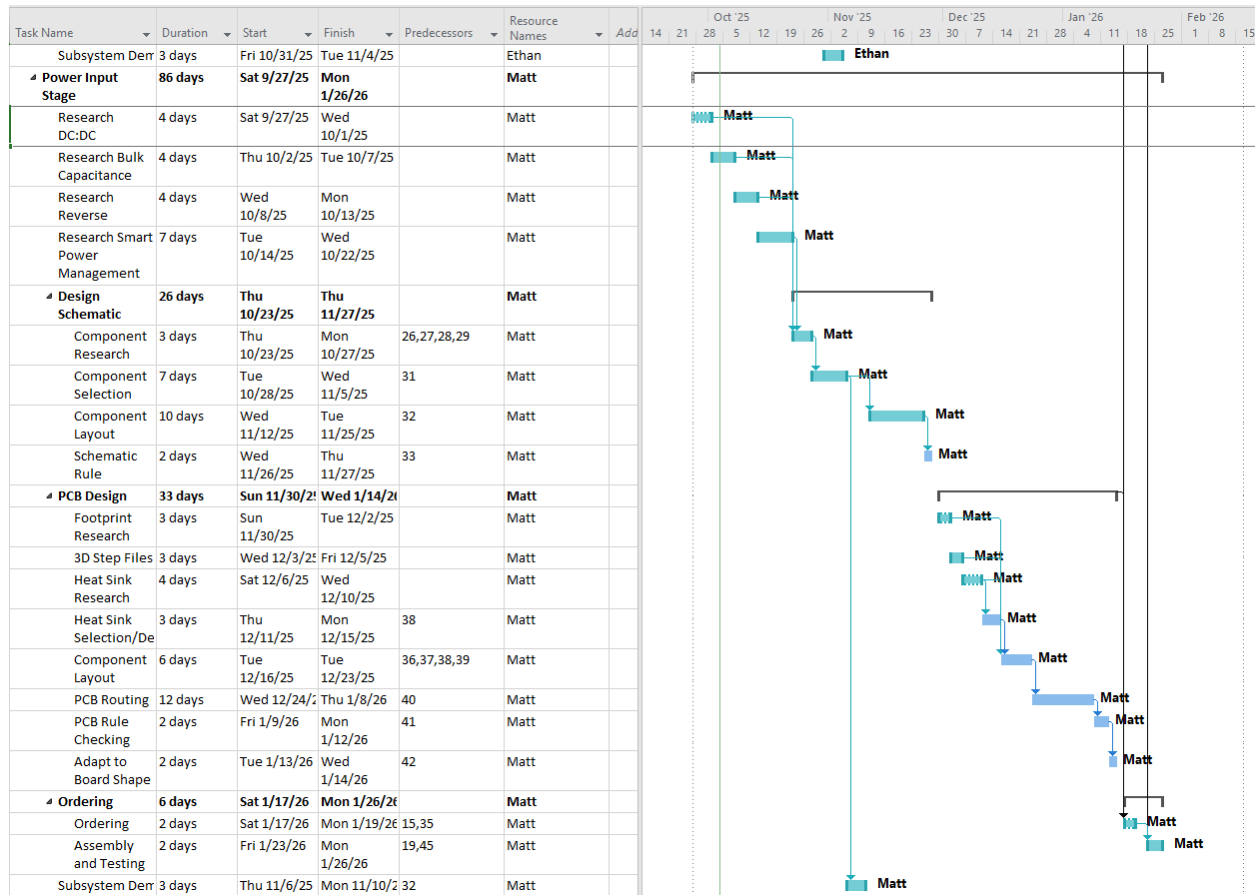


Figure 32: Power Input Stage Subsystem Gantt Chart

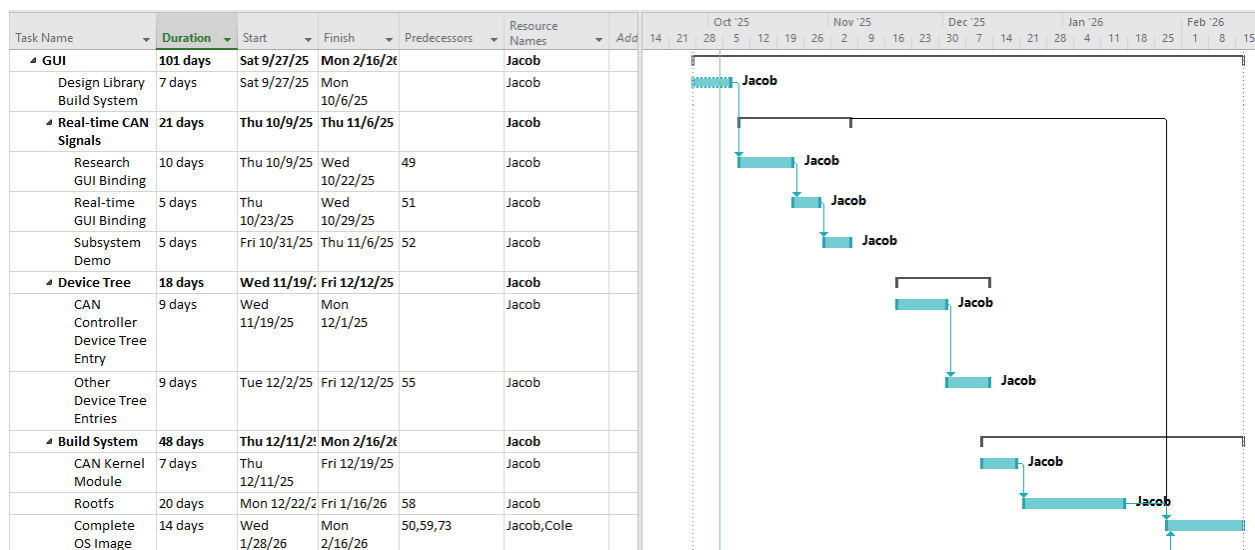


Figure 33: Graphical User Interface Subsystem Gantt Chart

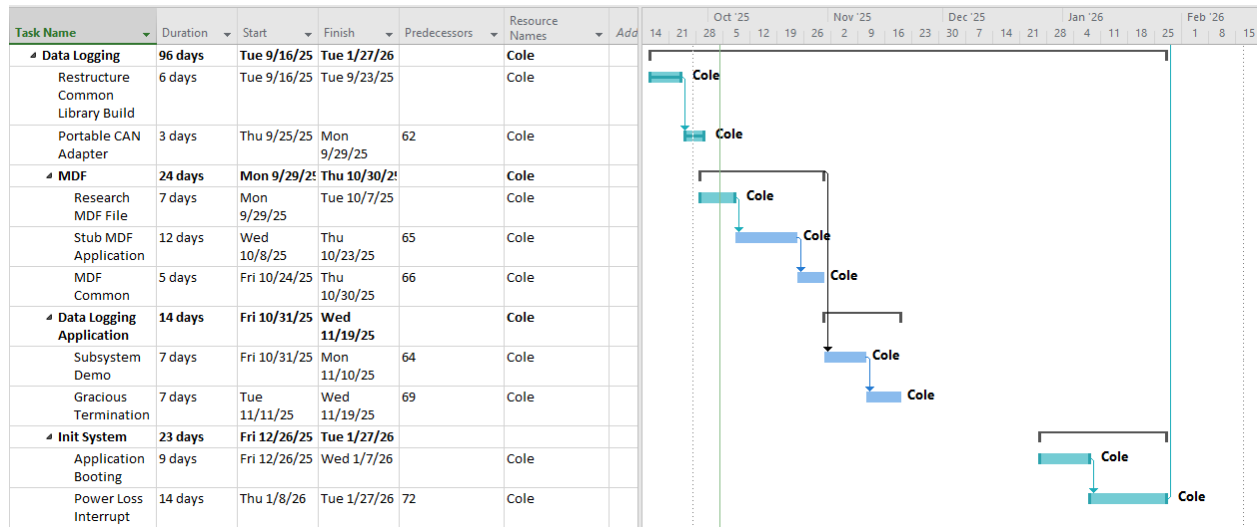


Figure 34: Data Logging Subsystem Gantt Chart

Table 27: Work Breakdown Structure Table

ID	Activity	Description	Deliverables / Checkpoints	Duration (days)	People	Resources	Predecessors
1	Embedded System						
1.1	Altium Research	Figure out differential classes and net pairs in Altium as well as impedance matching	Identify necessary Altium features to implement routing of HDMI and Ethernet	2	Ethan	Internet	
1.2	Design schematics						1.1
1.2.1	Component Selection	Select necessary components for all embedded circuitry	BOM of ICs	2	Ethan	Internet	1.1
1.2.2	Component Research	Read datasheets for chosen devices	Identify implantation of circuits	3	Ethan	Internet	1.2.1
1.2.3	Schematic Layout	Design embedded schematic	Complete schematic routing and implement all passives	21	Ethan	Altium	1.2.2
1.3	PCB Layout						
1.3.1	Board Shape	Determine board shape from the chosen display	DXF of circuit board	3	Ethan	Altium	1.2
1.3.2	Connector Placement	Place HDMI, USB, and interface connection.	Capture of PCB	7	Ethan	Altium	1.3.1
1.3.3	HDMI Bridge	Design HDMI to DMI Interface using either a custom male hdmi PCB or a ribbon cable interface	Capture of connections	3	Ethan	Altium	1.3.2

1.3.4	Component Placement	Place SoM and all ICs on the PCB	Capture of PCB	2	Ethan	Altium	1.3.3
1.3.5	Route PCB	Route all traces on the PCB to complete layout	Capture of PCB	18	Ethan	Altium	1.3.4
1.3.6	Rule Checking	Pass Altium's error checking and repairing all violations	Copy of DRC report and STP file of PCB	2	Ethan	Altium	1.3.5
1.4	Ordering				Ethan		
1.4.1	Gerber Generation	Generate gerber files for PCB	Uploaded gerber files	1	Ethan		1.3
1.4.2	PCB Order	Order PCB using gerber files	Receipt	1	Ethan		1.4.1
1.4.3	Component Order	Order all necessary circuit board components	Receipt	1	Ethan		1.4.2
1.5	Assembly and testing				Ethan		
1.5.1	Assemble PCB	Place all components on board and solder	Built circuit board	3	Ethan		1.4
1.5.2	Test High Frequency Components	Validate HDMI, Ethernet, SDIO, and USB functionality	Capture of interface working correctly	2	Ethan		1.5.1
1.5.3	Test Embedded components	Validate CANBUS	Capture of device interfacing with CAN bus	3	Ethan		1.5.2
1.5.4	Build interface wire harness	Design and make wire harness to connect the dashboard to the vehicle wire harness	Picture of harness	7	Ethan		1.5.3

1.6	Subsystem Demo	Connect all necessary circuits to the compute module to demonstrate Display and CAN functionality	Picuture of configuration and demonstration of the display and the device reading the CAN bus	3	Ethan		
2	Power Input Stage						
2.1	Research D/D converter	Figure out route of converting 11-28 VDC to 5 VDC	Identify and calculate needed components for conversion	4	Matt		
2.2	Research Bulk Capacitance for Shut Down on SOM	Calculate correct value of capacitance to allow SOM to correctly shut down. Also type of capacitor to use.	Select type of capacitors to use, size, and amount.	4	Matt		
2.3	Research Reverse Polarity to protect PCBs	Research into type of reverse polarity protection and component required.	Select components and circuit design to implement load.	4	Matt		
2.4	Research Smart Power Management	Research some power switching and signals for power on and power off.	Select needed components and finalize circuit design.	7	Matt		
2.5	Design Schematic						
2.5.1	Component Research	Research to assure components meet the necessary interfacing, ie		3	Matt		2.1, 2.2, 2.3, 2.4

		components work together					
2.5.2	Component Selection	Finalize list of components that will be used in design	BOM of ICs	7	Matt		2.5.1
2.5.3	Component Layout	Place IC and low level component on schematic	Schematic of ICs and lower level components' connections.	10	Matt		2.5.2
2.5.4	Schematic Rule Checking	Use built in tools to ensure all connections are properly implemented.	Minimum warnings or 0 Errors on the DCR	2	Matt		2.5.3
2.6	PCB Design						
2.6.1	Footprint Research	Connect the schematic component to their footprint counter parts	All components have a footprint	3	Matt		2.5
2.6.2	3D Step Files	Research into 3D Designs and connect them to footprints for 3D board assembly. Used for enclosure design.	At least high-level components have an appropriate 3D design file	3	Matt		
2.6.3	Heat Sink Research	Research into how to transfer power loss through heat away from the PCB.	Figure out what level heatsink and assure components have needed heat sink support.	4	Matt		
2.6.4	Heat Sink Selection/Design	Assure heat sinks can fit in board layout.	Component placements are measured out appropriately	3	Matt		2.6.3
2.6.5	Component Layout	Place footprints on	Capture of PCB	6	Matt		2.6.1, 2.6.2, 2.6.4

		PCB cut out representation					
2.6.6	PCB Routing	Apply trace routing and assure they of too size to meet current	Capture of PCB trace routes	12	Matt		2.6.5
2.6.7	PCB Rule Checking	Compare placement and traces routing meets design requirement.	Use built in tool to get zeros errors and minimum warnings.	2	Matt		2.6.6
2.6.8	Adapt to step 1.3.1 board shape	Assure layouts can fit on board design of pervious steps	Finalized Board Layout	2	Matt		2.6.7
2.7	Ordering						2.6
2.7.1	Corequisite combine with step 1.4 ordering	Combination of PCB, Components to pervious step's order	Combine order with SoM design	2	Matt		1.4
2.8	Corequisite assembly testing for step 1.5	Combine testing with pervious steps	Combine testing with SoM testing.	2	Matt		1.5
2.9	Subsystem Demo	Build a functional bread design of the subsystem for demonstrating powering of SoM	Physical bread built with through hole components.	3	Matt		2.6.7
3	GUI						
3.1	Design Library Build System	Toolchain for building and configuring the Dashboard GUI	Parse information from the CAN bus to the GUI	7	Jacob	GNU Compiler Collection	
3.2	Real-time CAN signals				Jacob		
3.2.1	Research GUI Bindings	Determine how elements of the GUI can	GUI element bound to C datatype	10	Jacob	GTK documentation	3.1

		be bound to backend data					
3.2.2	Real-time GUI Binding	Binding element to backend data in real-time	Widget that can display real-time value of a C datatype	5	Jacob	GTK documentation	3.2.1
3.2.3	Real-time CAN signals (subsystem demo)	Binding element to CAN signal in real-time	Widget that can display real-time value of a CAN signal	5	Jacob	GTK documentation, Internal documentation	3.2.2
3.3	Device Tree						
3.3.1	CAN controller device tree entry	Define the device tree entry for the SPI based CAN controller	SocketCAN device enumeration	9	Jacob	Raspberry PI Documentation	
3.3.2	Other device tree entries	Any required device configuration for SoM hardware	Functional control of EMMC, display, touchscreen	9	Jacob	GNU Compiler Collection The Device Tree Project	3.3.1
3.4	Build System						
3.4.1	CAN kernel module	Build SocketCAN kernel module into OS kernel	Built kernel with SocketCAN	7	Jacob	Buildroot documentation	
3.4.2	Rootfs	System for building and configuring Linux kernel and building OS image	Built OS image for SoM, hooks for application binaries	20	Jacob	Buildroot documentation	3.4.1
3.4.3	Complete OS image	Complete system for building OS image	OS image including device tree, built kernel, and application binaries	14	Jacob, Cole	Buildroot documentation, Internal documentation	3.2, 3.4.2, 4.5

4	Data Logging						
4.1	Restructure common library toolchain	GUI and data logging functionality will be part of the common library	Subdirectories for GUI and data logging applications. Linkage with GUI library.	6	Cole	Internal documentation	
4.2	Portable CAN adapter interface	Interface for CAN bus must be implemented for multiple OS's	Functional implementation for Linux x86/64, Linux ARM, and Windows x86/64	3	Cole	Internal documentation, Socket-CAN documentation, SLCAN documentation	4.1
4.3	MDF Standard						
4.3.1	Research MDF file format	Data logs should use MDF file format	Understanding of internal structure, ability to create from scratch	7	Cole	ASAM MDF standard	
4.3.2	Stub MDF application	Application to test writing info into MDF format	Application that can write simple data into the MDF format	12	Cole	ASAM MDF standard	4.3.1
4.3.3	MDF common library	Functionality for working with MDF files should be moved into common library	Set of functions for creating and appending data to MDF files	5	Cole	Internal documentation	4.3.2
4.4	Data logging application						
4.4.1	Basic data logging (subsystem demo)	Using MDF tools, CAN traffic should be logged to a file	Application capable of creating an MDF data log of CAN traffic	7	Cole	Internal documentation, POSIX Man pages,	4.3

						C99 standard	
4.4.2	Gracious termination	Well defined termination sequence	Data logging application should terminate gracefully when signaled. Data log is not corrupted	7	Cole	Internal documentation, POSIX man pages, C99 standard	4.4.1
4.5	Init system						
4.5.1	Application booting	System for booting the data logging and GUI applications	Programmatic method of launch applications with specified arguments	9	Cole	POSIX man pages, C99 standard	
4.5.2	Power loss interrupts	Function for detecting and reacting to power loss events	Triggering of event based on GPIO interrupt	14	Cole	Raspberry Pi documentation, POSIX man pages	4.5.1

9. Conclusions and Recommendations (CB)

While this project, in terms of senior design, is mostly finished meeting all initial goals, there are still improvements that are intended to be made in the near future for Zips Racing's design cycle. The device's waterproofing is one such improvement, as currently, the device on its own is not waterproof. For the sake of Zips Racing's competition at Formula Hybrid+Electric, the device was sealed externally with silicon. While this waterproofing was very successful (it withstood several hours in the rain at competition) it leaves much to be desired in terms of maintainability and aesthetics. A more elegant solution would be to embed a gasket into the enclosure design, both where the two halves of the clamshell meet and where the screen meets the enclosure itself. This, combined with the already waterproof Autosport connector, should eliminate any concerns for water damage. In terms of software, the flexibility of the GUI needs more improvement. While using the CAN DBC standard allows changes to be made to the encoding of information, the GUI currently does not allow the adding or removing sources of information. For instance, if a new sensor is added to the vehicle, the source code of the GUI needs modified to add said information to it. Preferably, a configuration file could be modified to add the new sensor to the GUI.

In terms of long-term improvements, the device's mechanical packaging is the most obvious. Switching from HDMI to an TFT with an external controller for the display communication allows replacing the HDMI bridge with a ribbon cable. In addition to improving the durability (ribbon cables cannot back out), this also would allow the board stack to be much slimmer. Further, there are always new features and optimizations to the layout of the GUI that could be made. Such changes are left to the non-graduating members of Zips Racing.

10. References

- (1) Benedetti, D., Agnelli, J., Gagliardi, A., Dini, P., & Saponara, S. (2020). *Design of a Digital Dashboard on Low-Cost Embedded Platform in a Fully Electric Vehicle*. IEEE.
https://www.researchgate.net/profile/Pierpaolo-Dini/publication/343496549_Design_of_a_Digital_Dashboard_on_Low-Cost_Embedded_Platform_in_a_Fully_Electric_Vehicle/links/5f33c6e792851cd302ef5c99/Design-of-a-Digital-Dashboard-on-Low-Cost-Embedded-Platform-in-a-Fully-Electric-Vehicle.pdf
- (2) Nimmala, R., & Nimmala, J. (2024). *Racing Into the Data Age: Sensor Intelligence, Advanced Analytics, and Kafka in Formula 1 Race Car*. International Journal of Artificial Intelligence & Machine Learning.
https://iaeme.com/MasterAdmin/Journal_uploads/IJAIML/VOLUME_3_ISSUE_1/IJAI ML_03_01_006.pdf
- (3) Györök, G., Beszédes, B. (2025). Highly reliable data logging in embedded systems.
<https://core.ac.uk/download/pdf/154883632.pdf>
- (4) Brewu, D. (2022). *Design and Implementation of a CAN BUS Data Logger with Multiple Communication Interfaces for Vehicles*. Ashesi University.
<https://air.ashesi.edu.gh/server/api/core/bitstreams/45c51ae8-08c9-463e-9bc0-28e8ec2220f0/content>
- (5) Drew, M. & Herron, B. (2010). *Data Acquisition and Display System for Motor Vehicle* (U.S. Patent No. 7,786,851). Opus IVS Inc.
- (6) Damiani, S., Deregibus, E., & Dosio, N. (2003). *Vehicle Display Device* (U.S. Patent No. 6,667,726). Centro Ricerche Fiat SCpA.
- (7) Curry, A., Clarke, S., & Samuel, Aamod (2023). *X-57 Cockpit Display System Development and Features*. NASA Technical Reports Server.
<https://ntrs.nasa.gov/api/citations/20230005312/downloads/20230005312FINAL.pdf>
- (8) Corrigan, S. (2016). *Introduction to the Controller Area Network. (CAN)*. Texas Instruments.
<https://www.ti.com/lit/an/sloa101b/sloa101b.pdf>

11. Appendices

Appendix A

Third-Party Standards, Resources, and Datasheets

Raspberry Pi Compute Module 5 Datasheet. <https://datasheets.raspberrypi.com/cm5/cm5-datasheet.pdf>

MCP25625 Datasheet.

<https://ww1.microchip.com/downloads/aemDocuments/documents/OTH/ProductDocuments/DataSheets/MCP25625-CAN-Controller-Data-Sheet-20005282C.pdf>

POSIX Manual Pages. <https://man7.org/linux/man-pages/index.html>

Linux GPIO Driver. <https://www.kernel.org/doc/html/latest/driver-api/gpio/driver.html>

Linux PWM Interface. <https://docs.kernel.org/driver-api/pwm.html>

Linux SocketCAN API. <https://www.kernel.org/doc/html/latest/networking/can.html>

ASAM MDF Standard. <https://www.asam.net/standards/detail/mdf/>

ASAM MDF Library Documentation <https://asammdf.readthedocs.io/en/latest/index.html>

GTK 4 Library Documentation. <https://docs.gtk.org/gtk4/>

Introduction to the CAN DBC File. <https://www.csselectronics.com/pages/can-dbc-file-database-intro>

DBC Database File Format <https://docs.fileformat.com/database/dbc/>

Appendix B

First-Party Source Code

Due to the size and complexity of this project's software, no source code is directly included in this report. All software developed for this design project is publicly available on GitHub. The URL of the project's repository and specific sections of the code referenced in this report are included in the table below.

Resource	GitHub URL
Repository Root & Documentation	https://github.com/ZipsRacingElectric/DART-ZR
CAN Device Library	https://github.com/ZipsRacingElectric/ZRE-CAN-Tools/tree/main/lib/can_device
CAN Database Library	https://github.com/ZipsRacingElectric/ZRE-CAN-Tools/tree/main/lib/can_database
MDF Library	https://github.com/ZipsRacingElectric/ZRE-CAN-Tools/tree/main/lib/mdf
MDF Bus Logging Specification	https://github.com/ZipsRacingElectric/ZRE-CAN-Tools/blob/main/lib/mdf/mdf_can_bus_logging.h
GUI Application	https://github.com/ZipsRacingElectric/ZRE-CAN-Tools/blob/dashboard_gui_jk/src/dashboard_gui_jk/main.c
Data Logging Application	https://github.com/ZipsRacingElectric/ZRE-CAN-Tools/blob/main/src/can_mdf_logger/main.c
Init-system	https://github.com/ZipsRacingElectric/DART-ZR/blob/main/src/init_system/main.c